Software Quality Engineering

Lecture 13

CI using Jenkins

Instructor:

Dr. Natalia Chaudhry,

Assistant Professor, Software Engineering Department, PU.

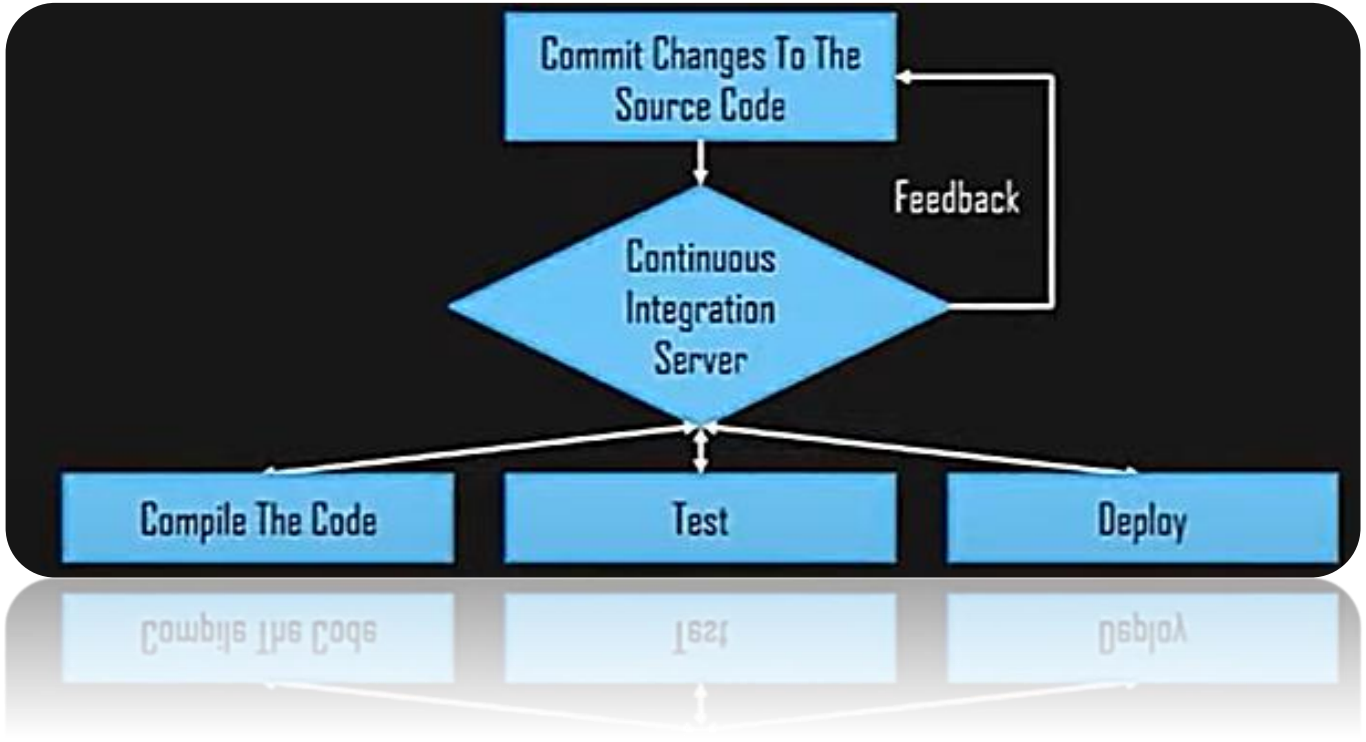
Agenda

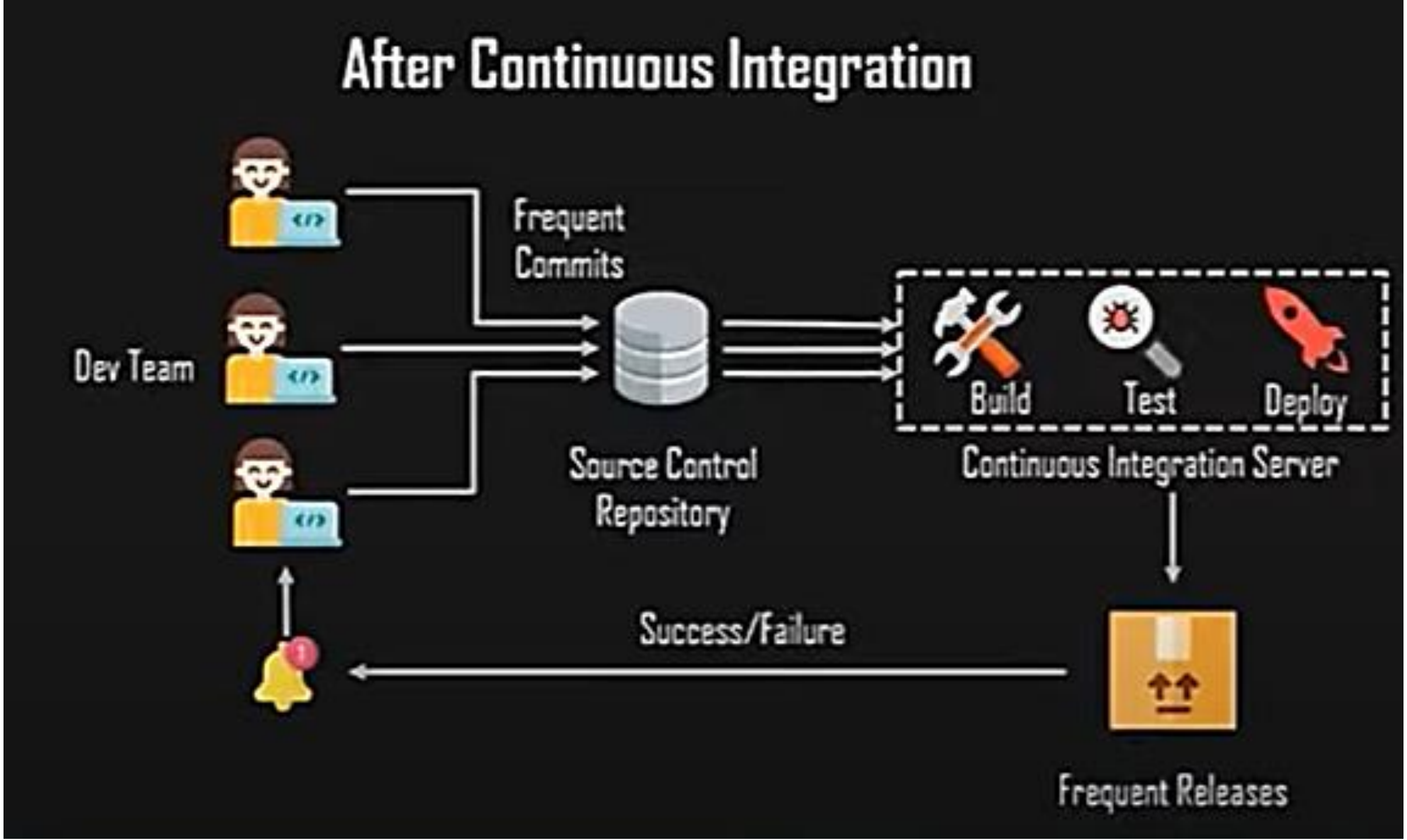
- Continuous integration with Jenkins



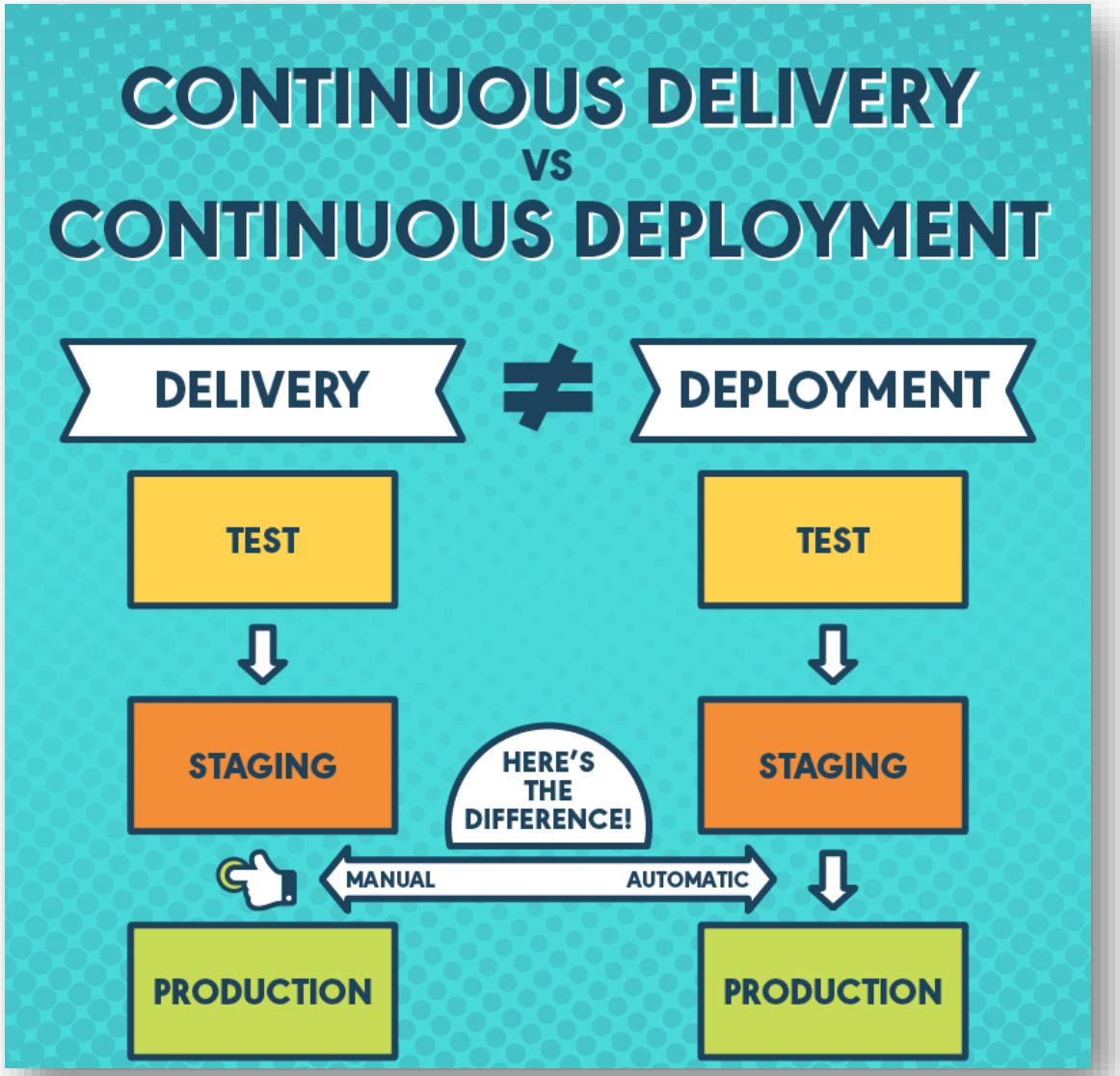
CI

It lets the developers to commit changes to the source code in shared repository several times a day or more frequent

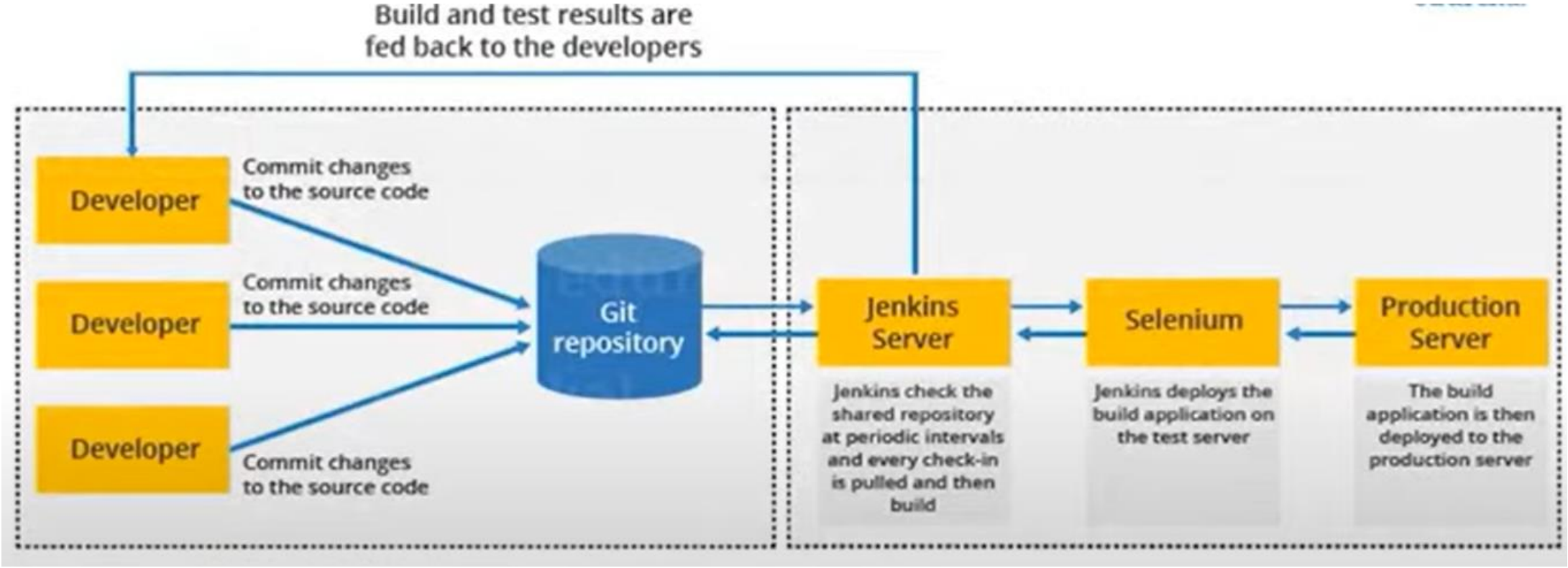



After Continuous Integration
Frequent
Commits
Dev Team
Source Control
Repository
Build
Test
Deploy
Continuous Integration Server
Success/Failure
Frequent Releases

Continuous delivery vs continuous deployment



Jenkins architecture



How the architecture works?

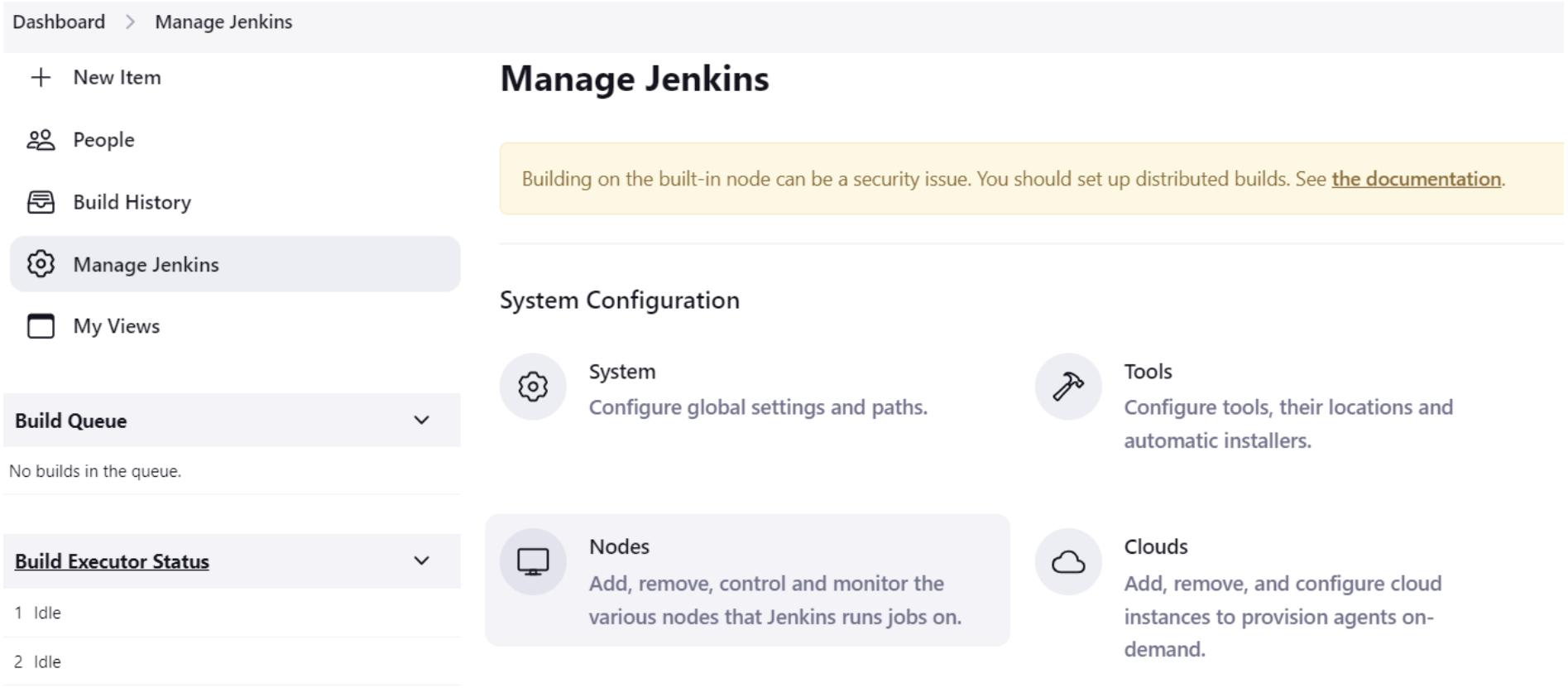
Master slave architecture…

- Jenkins master uses multiple slaves (for various environments such as ubuntu windows etc) for testing and generate test report.

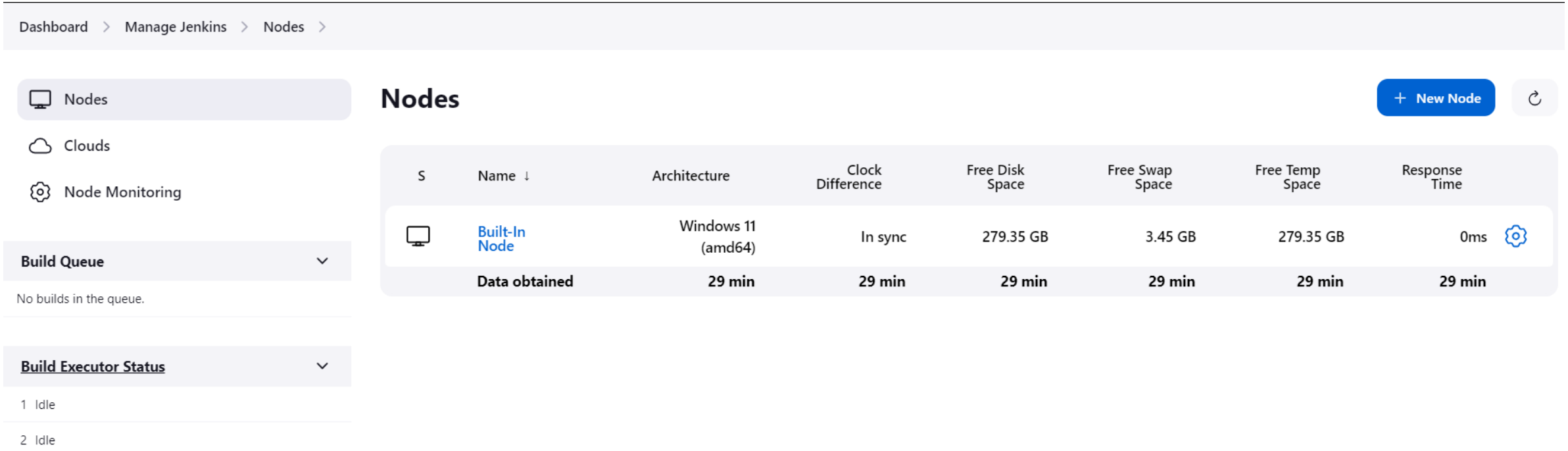

demo

Add slave

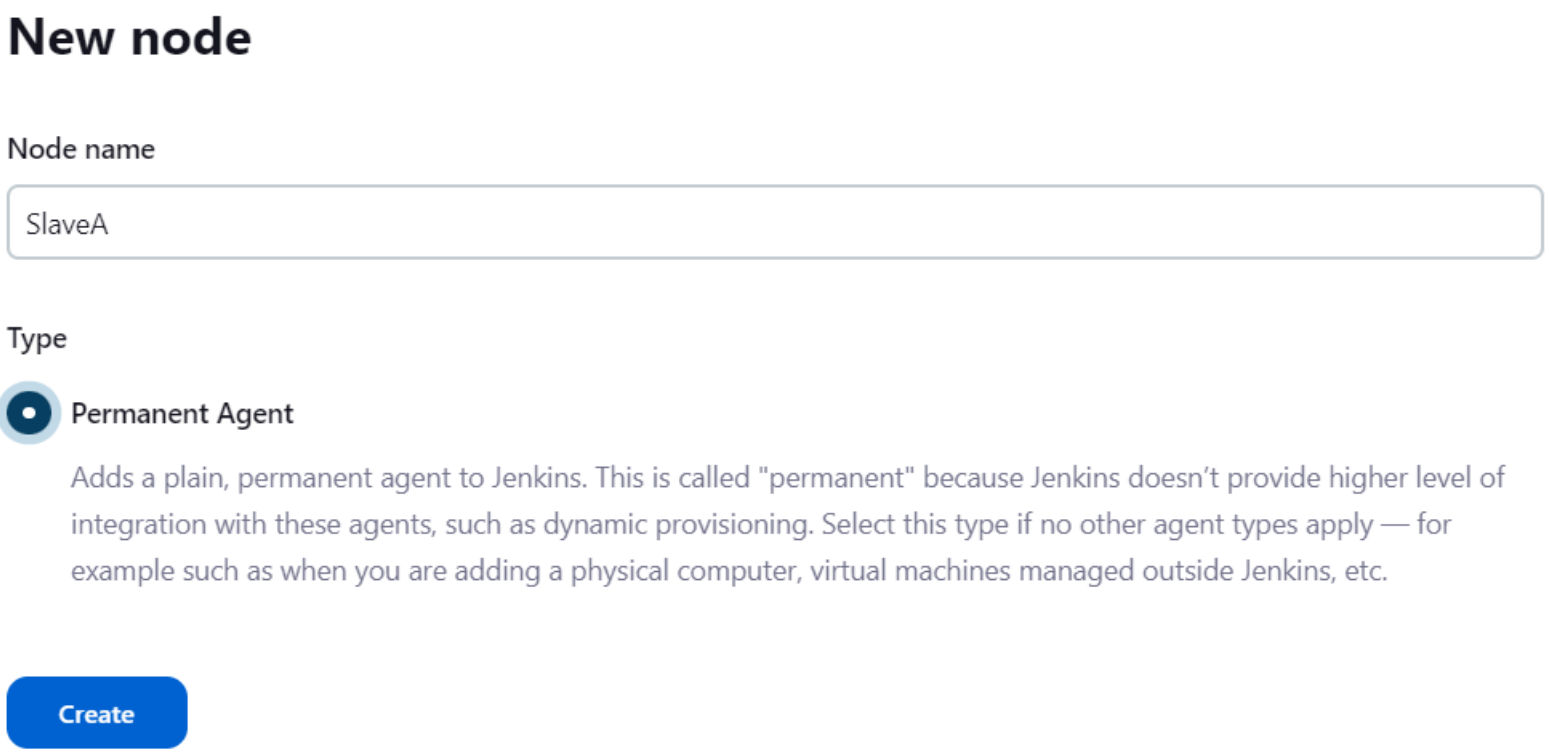
- Move to Jenkins homepage/dashboard
- http://localhost:8080/
- Go to **Manage Jenkins->Nodes**



- Select **New Node**



- Name it **SlaveA** or any other name
- Check **permanent agent** and **Create**

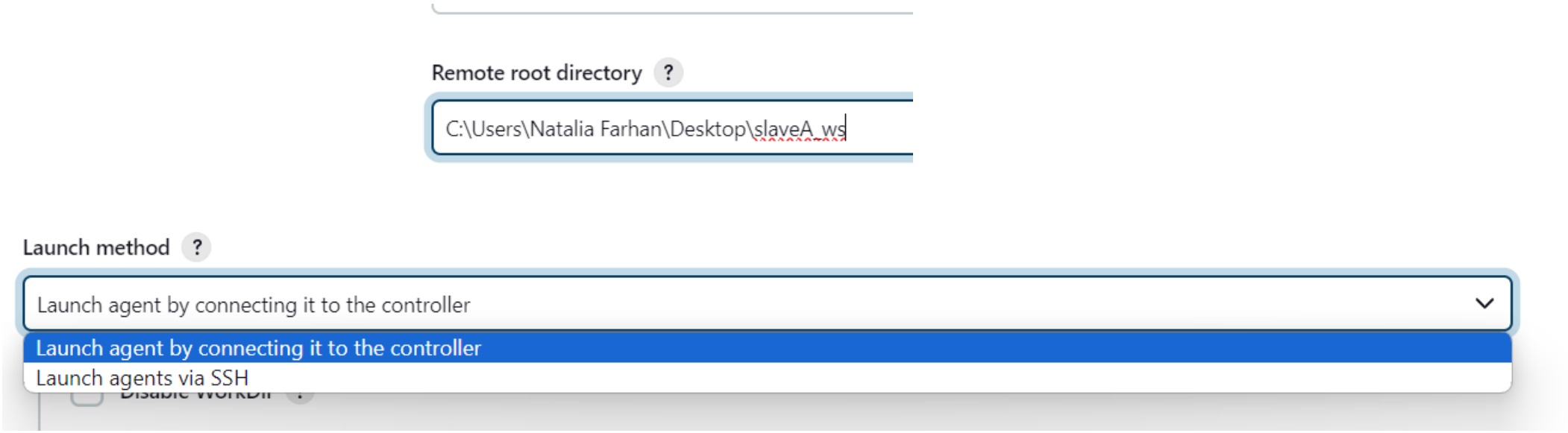


Labels represent a way of naming one or more slaves. We leverage this labeling system to tie the execution of a job directly to one or more slave nodes.

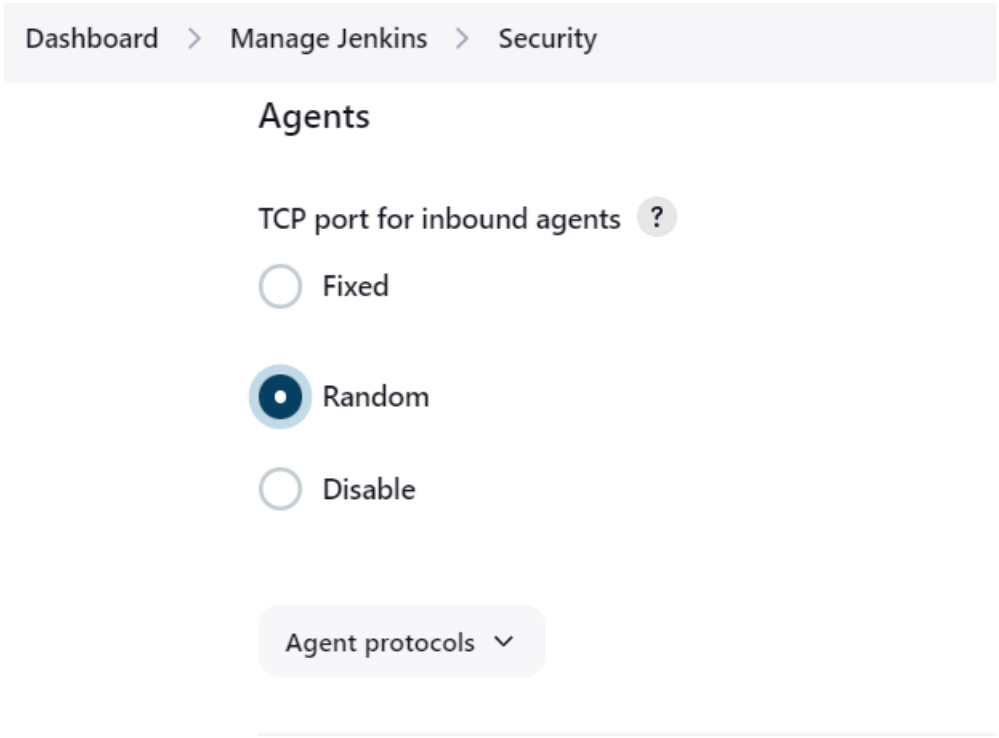
powerful load-balanced Jenkins solutions

- When Jenkins discovers a job execution is pending, which is tied to a label, it will attempt to locate any available slave nodes tagged with that label that are not in use.
- If any nodes with that label are free Jenkins will run the job on the available node.
- If no nodes are available, Jenkins will queue the job for the next available node that has the specified .

- No of executers is the number of jobs a slave is able to run in parallel way
- Remote root directory
  - Create workspace for slave. For that create a folder named **slaveA_ws** on your PC
  - Place location path of this folder in Jenkins **remote root directory**



- To get option in launch method “…connect to master”. Goto dashboard->manage jenkins.-> security-> put option as **Random**



- Launch by typing following commands for Windows (copy/paste) on cmd

Agent SlaveA

Mark this node temporarily offline

Jenkins Slave

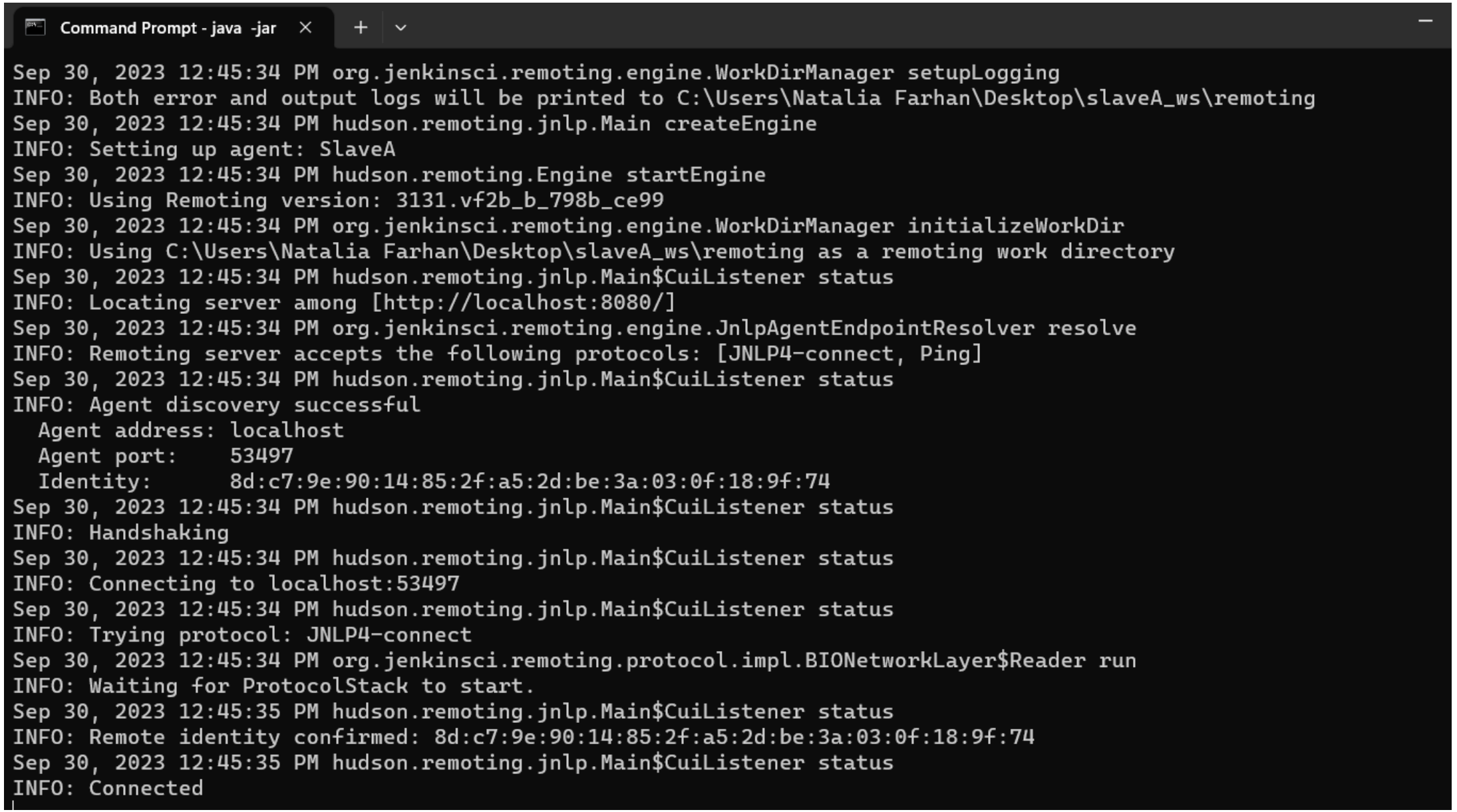
Edit description

Run from agent command line: (Unix)

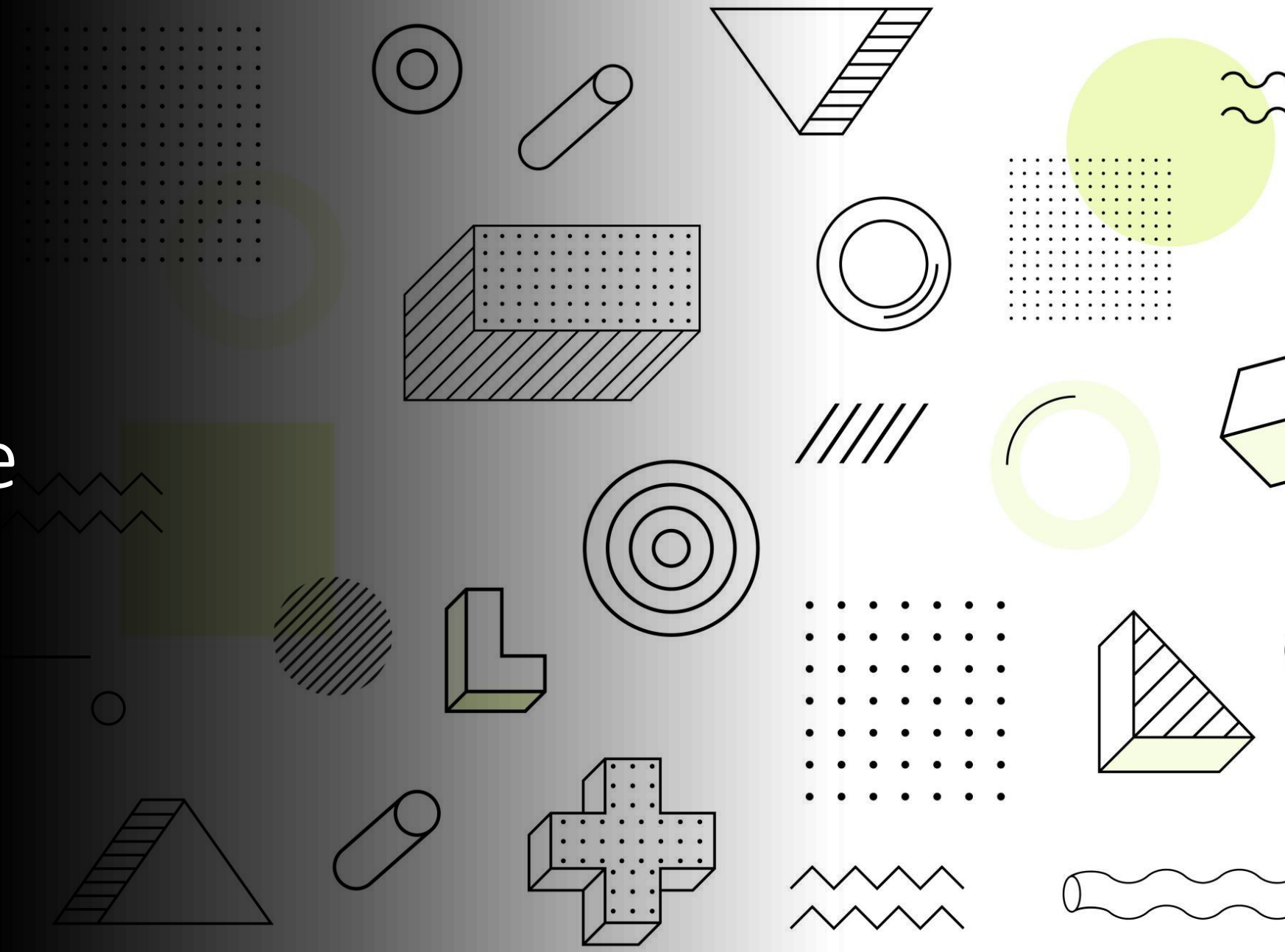
```
curl -sO http://localhost:8080/jnlpJars/agent.jar
java -jar agent.jar -jnlpUrl http://localhost:8080/computer/SlaveA/jenkins-agent.jnlp -secret
3911d2663ed1ea5b9e99e6eebd4b479ee9ef21582ec89cbb3358bf7b11620381 -workDir "C:\Users\Natalia Farhan\Desktop\slaveA_ws"
```

Run from agent command line: (Windows)

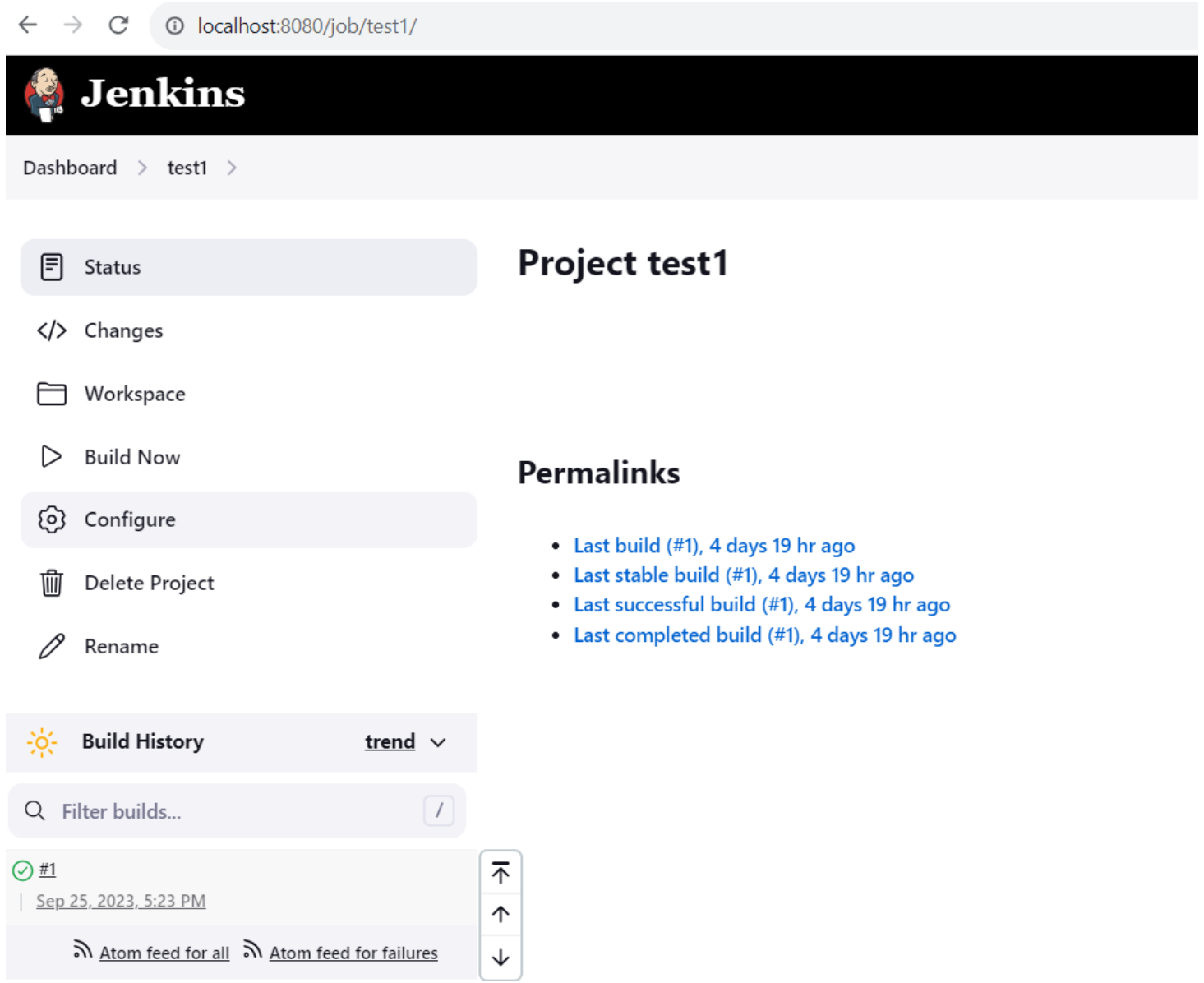
```
curl.exe -sO http://localhost:8080/jnlpJars/agent.jar
java -jar agent.jar -jnlpUrl http://localhost:8080/computer/SlaveA/jenkins-agent.jnlp -secret
3911d2663ed1ea5b9e99e6eebd4b479ee9ef21582ec89cbb3358bf7b11620381 -workDir "C:\Users\Natalia Farhan\Desktop\slaveA_ws"
```


Command Prompt - java -jar
Sep 30, 2023 12:45:34 PM org.jenkinsci.remoting.engine.WorkDirManager setupLogging
INFO: Both error and output logs will be printed to C:\Users\Natalia Farhan\Desktop\slaveA_ws\remoting
Sep 30, 2023 12:45:34 PM hudson.remoting.jnlp.Main createEngine
INFO: Setting up agent: SlaveA
Sep 30, 2023 12:45:34 PM hudson.remoting.Engine startEngine
INFO: Using Remoting version: 3131.vf2b_b_798b_ce99
Sep 30, 2023 12:45:34 PM org.jenkinsci.remoting.engine.WorkDirManager initializeWorkDir
INFO: Using C:\Users\Natalia Farhan\Desktop\slaveA_ws\remoting as a remoting work directory
Sep 30, 2023 12:45:34 PM hudson.remoting.jnlp.Main$CuiListener status
INFO: Locating server among [http://localhost:8080/]
Sep 30, 2023 12:45:34 PM org.jenkinsci.remoting.engine.JnlpAgentEndpointResolver resolve
INFO: Remoting server accepts the following protocols: [JNLP4-connect, Ping]
Sep 30, 2023 12:45:34 PM hudson.remoting.jnlp.Main$CuiListener status
INFO: Agent discovery successful
Agent address: localhost
Agent port: 53497
Identity: 8d:c7:9e:90:14:85:2f:a5:2d:be:3a:03:0f:18:9f:74
Sep 30, 2023 12:45:34 PM hudson.remoting.jnlp.Main$CuiListener status
INFO: Handshaking
Sep 30, 2023 12:45:34 PM hudson.remoting.jnlp.Main$CuiListener status
INFO: Connecting to localhost:53497
Sep 30, 2023 12:45:34 PM hudson.remoting.jnlp.Main$CuiListener status
INFO: Trying protocol: JNLP4-connect
Sep 30, 2023 12:45:34 PM org.jenkinsci.remoting.protocol.impl.BIONetworkLayer$Reader run
INFO: Waiting for ProtocolStack to start.
Sep 30, 2023 12:45:35 PM hudson.remoting.jnlp.Main$CuiListener status
INFO: Remote identity confirmed: 8d:c7:9e:90:14:85:2f:a5:2d:be:3a:03:0f:18:9f:74
Sep 30, 2023 12:45:35 PM hudson.remoting.jnlp.Main$CuiListener status
INFO: Connected

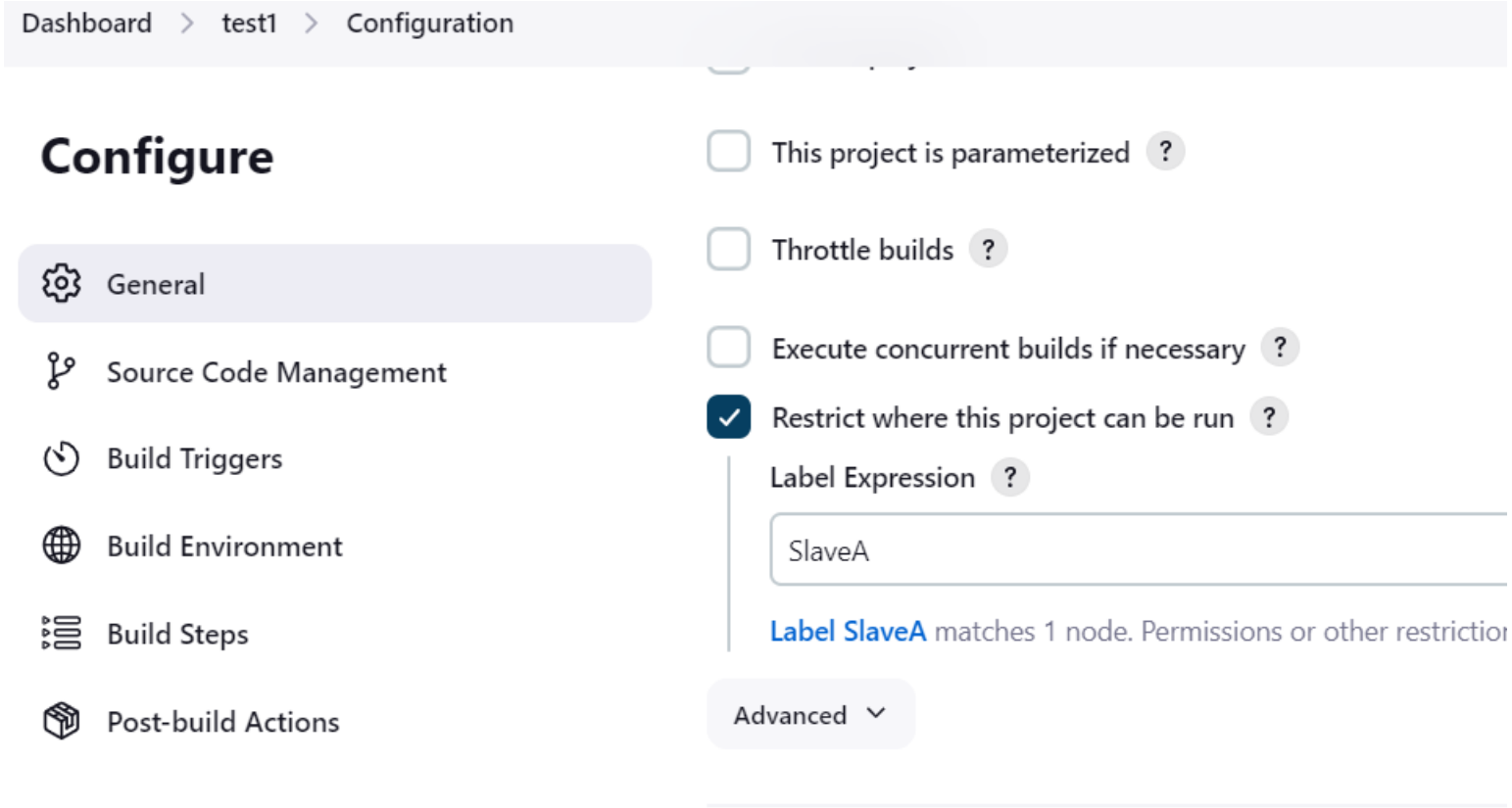
Configure master and slave on same machine



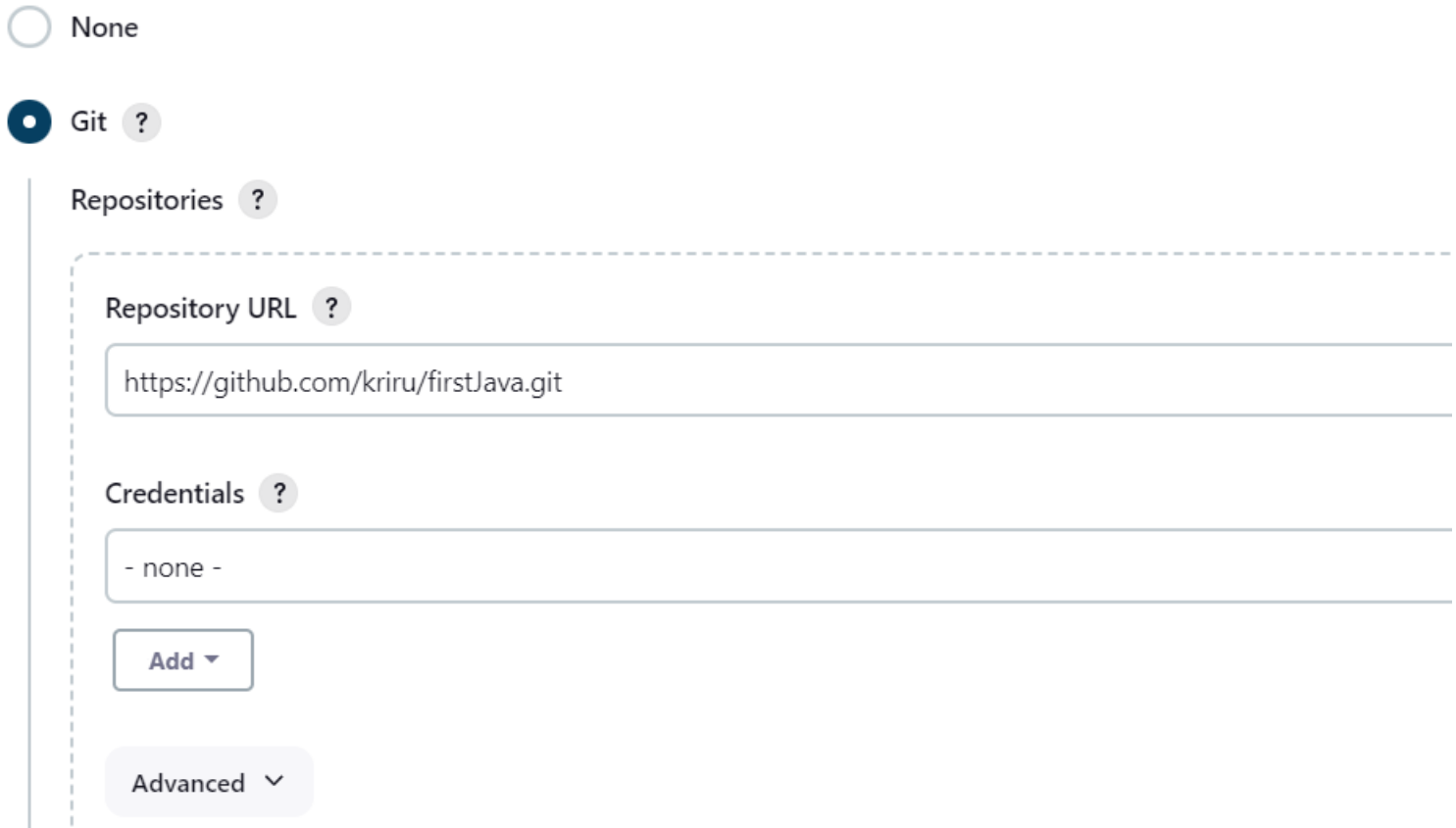
Configure already created job (last lecture) and select **Configure**



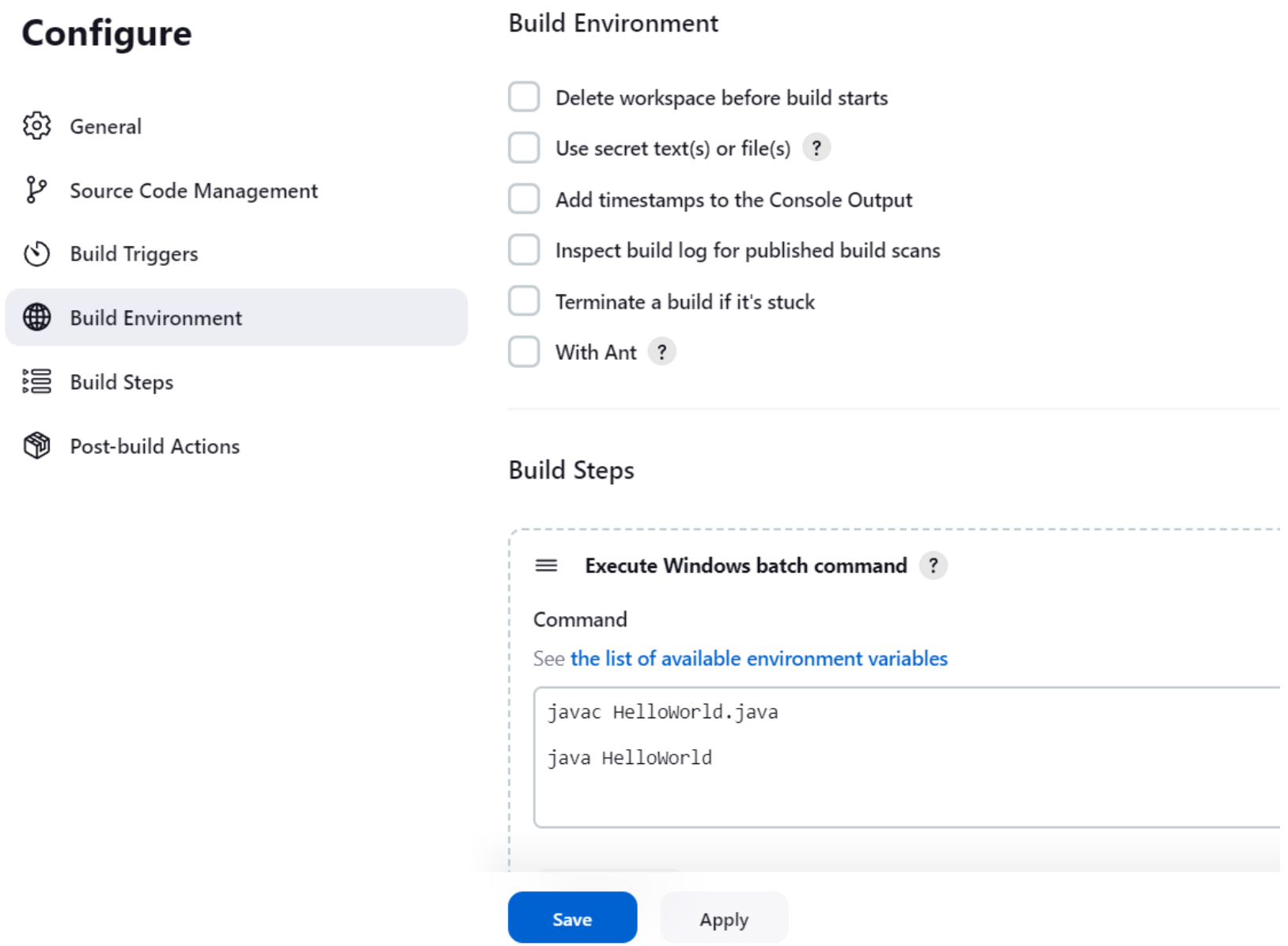
Select **Restrict where this project can be run**



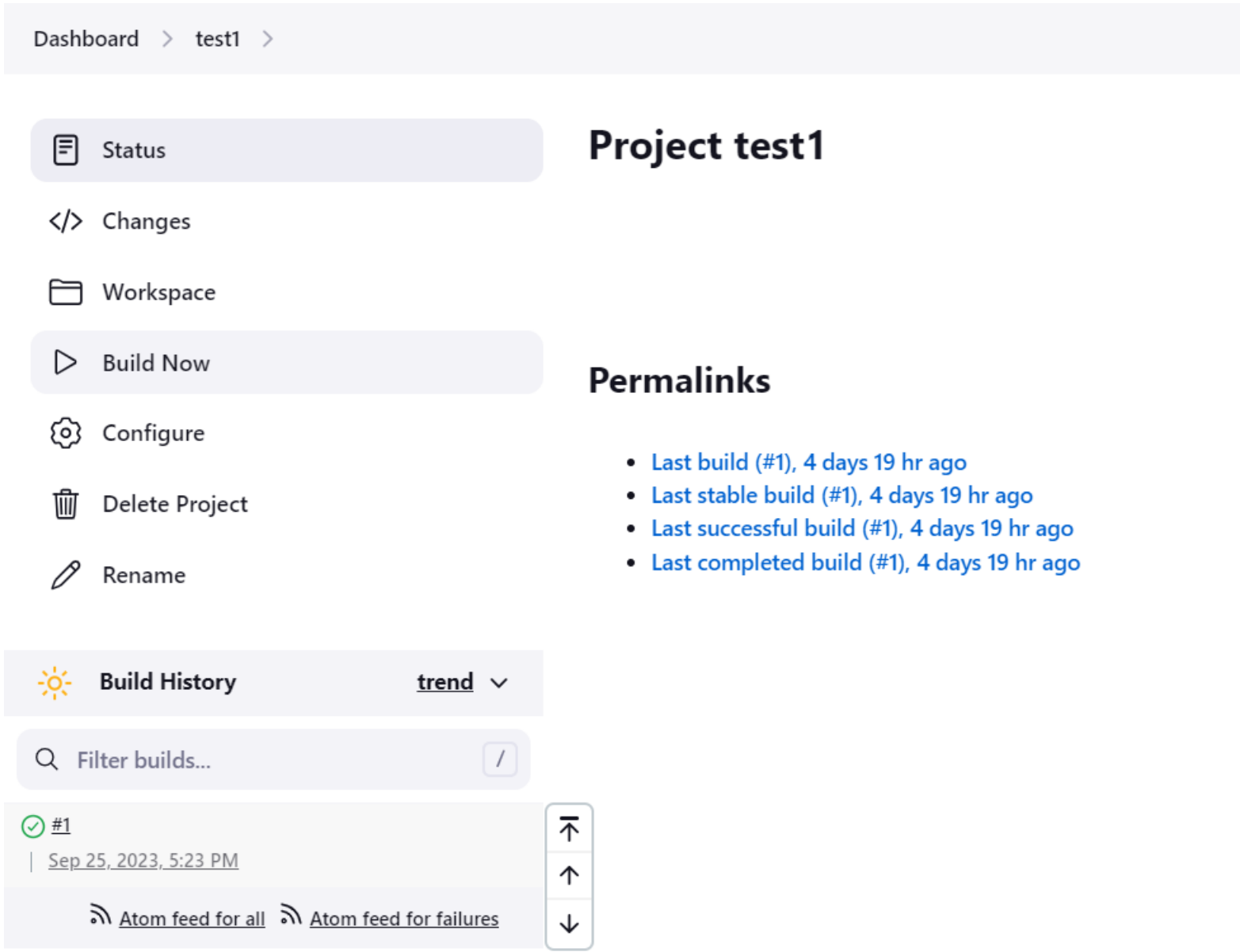
https://github.com/kriru/firstJava.git

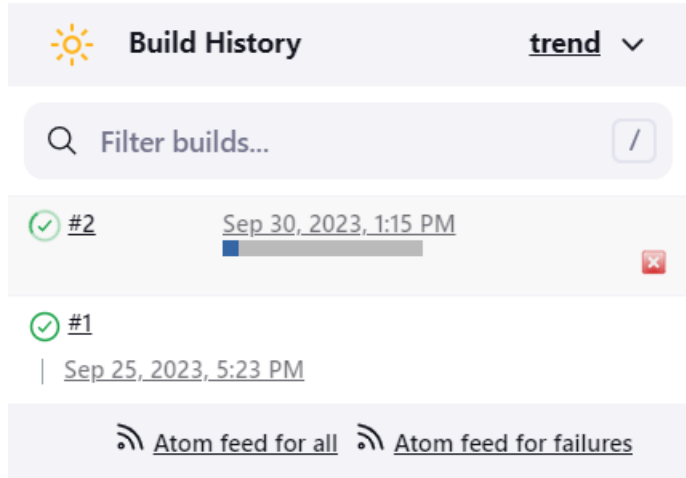


Click on apply



Build now


Dashboard
test1
Status
Changes
Workspace
Build Now
Configure
Delete Project
Rename
Project test1
Permalinks
Last build (#1), 4 days 19 hr ago
Last stable build (#1), 4 days 19 hr ago
Last successful build (#1), 4 days 19 hr ago
Last completed build (#1), 4 days 19 hr ago
Build History
trend
Filter builds...
/
#1
Sep 25, 2023, 5:23 PM
Atom feed for all
Atom feed for failures


Build History
trend
Filter builds...
/
#2
Sep 30, 2023, 1:15 PM
#1
Sep 25, 2023, 5:23 PM
Atom feed for all
Atom feed for failures

Job executed

Status
Changes
Console Output
View as plain text
Edit Build Information
Delete build '#2'
Git Build Data
Previous Build

Console Output

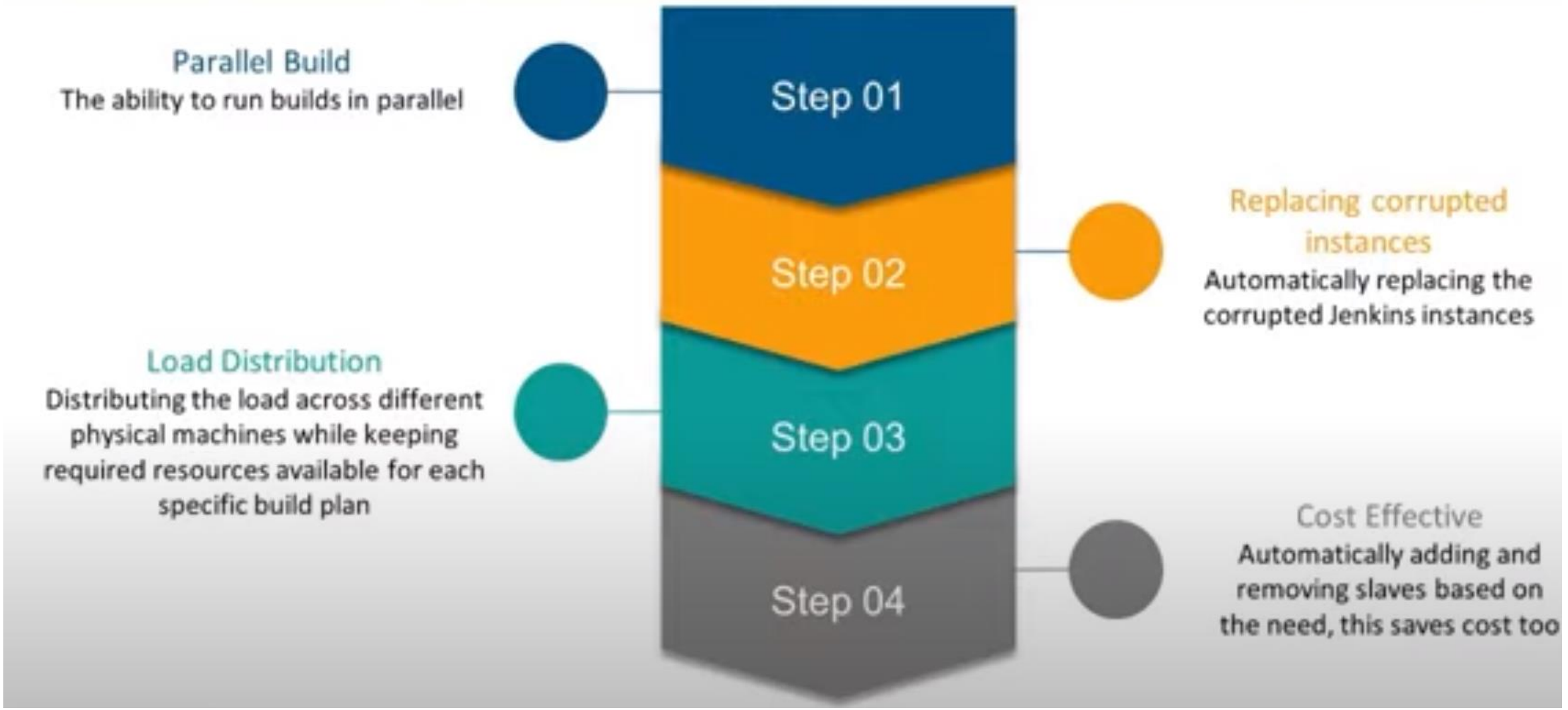
```
Started by user natalia
Running as SYSTEM
Building remotely on SlaveA (A) in workspace C:\Users\Natalia Farhan\Desktop\slaveA_ws\workspace\test1
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/kriru/firstJava.git
 > git.exe init C:\Users\Natalia Farhan\Desktop\slaveA_ws\workspace\test1 # timeout=10
Fetching upstream changes from https://github.com/kriru/firstJava.git
 > git.exe --version # timeout=10
 > git --version # 'git version 2.42.0.windows.2'
 > git.exe fetch --tags --force --progress -- https://github.com/kriru/firstJava.git +refs/heads/*:refs/remotes/origin/* # timeout=10
 > git.exe config remote.origin.url https://github.com/kriru/firstJava.git # timeout=10
 > git.exe config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
 > git.exe rev-parse "refs/remotes/origin/master^{commit}" # timeout=10
Checking out Revision 01a47017cd52f221fcaa157e1597fa3fb72b2267 (refs/remotes/origin/master)
 > git.exe config core.sparsecheckout # timeout=10
 > git.exe checkout -f 01a47017cd52f221fcaa157e1597fa3fb72b2267 # timeout=10
Commit message: "Update HelloWorld.java"
 > git.exe rev-list --no-walk 01a47017cd52f221fcaa157e1597fa3fb72b2267 # timeout=10
[test1] $ cmd /c call C:\Users\NATALI~1\AppData\Local\Temp\jenkins9144755125869583015.bat

C:\Users\Natalia Farhan\Desktop\slaveA_ws\workspace\test1>javac HelloWorld.java

C:\Users\Natalia Farhan\Desktop\slaveA_ws\workspace\test1>java HelloWorld
Hello, World
```

Jenkins scalability

- Working on multiple jobs simultaneously needs scalability



Distributed Jenkins Build

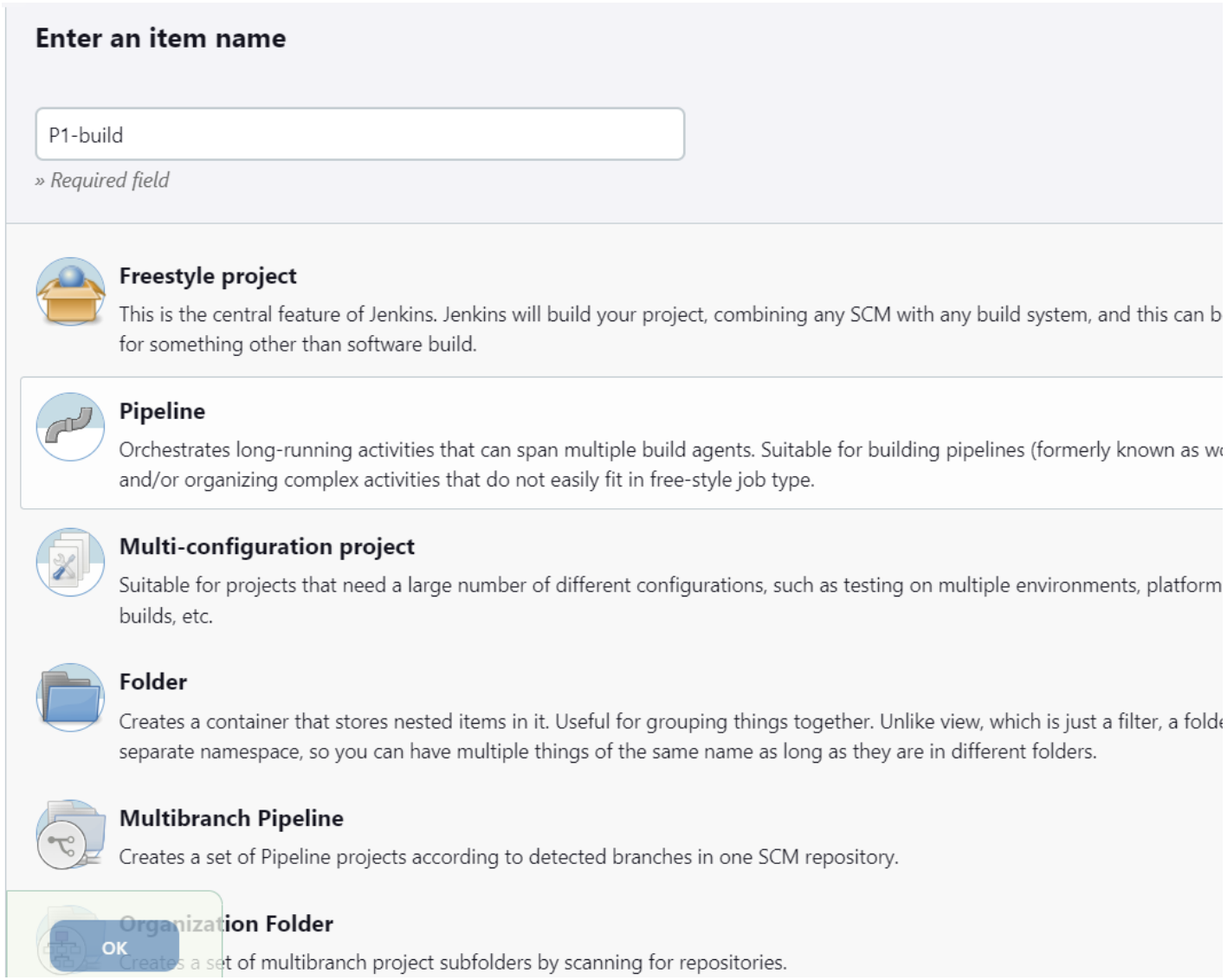
- To run heavy projects which gets build on regular basis on a single machine is not a good option
- We need to offload load from master by configuring more slaves
- Jenkins master will distribute load to slaves
- Master and slaves communicate using tcp/ip protocol

Pipeline

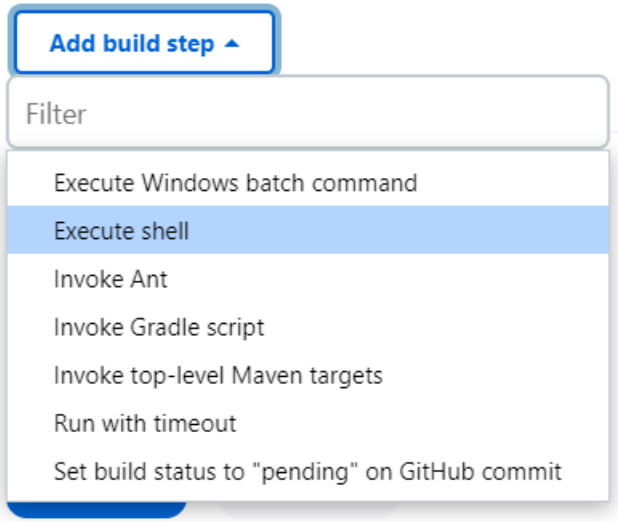
- Build Pipeline plugin are collection of Jenkins jobs.

Creating simple pipeline using pipeline plugin w/o jenkinsfile

Create job 1
****Freestyle project**

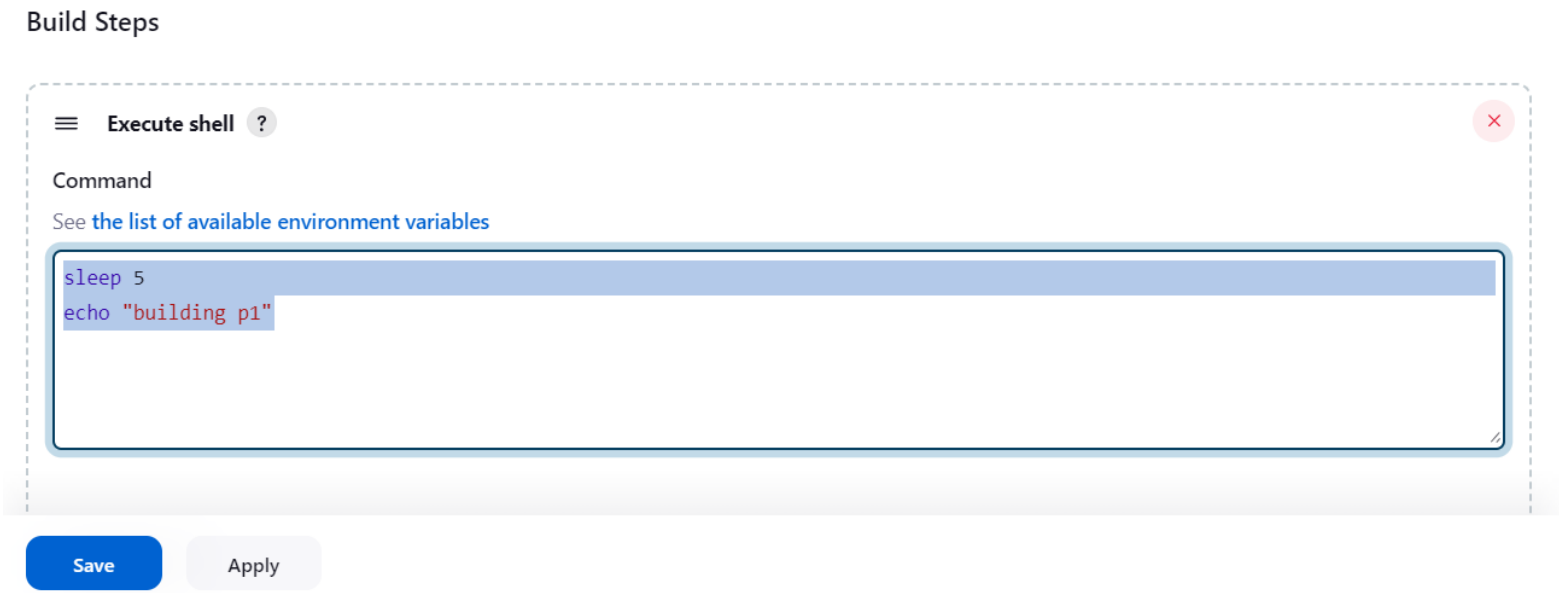


Build Steps

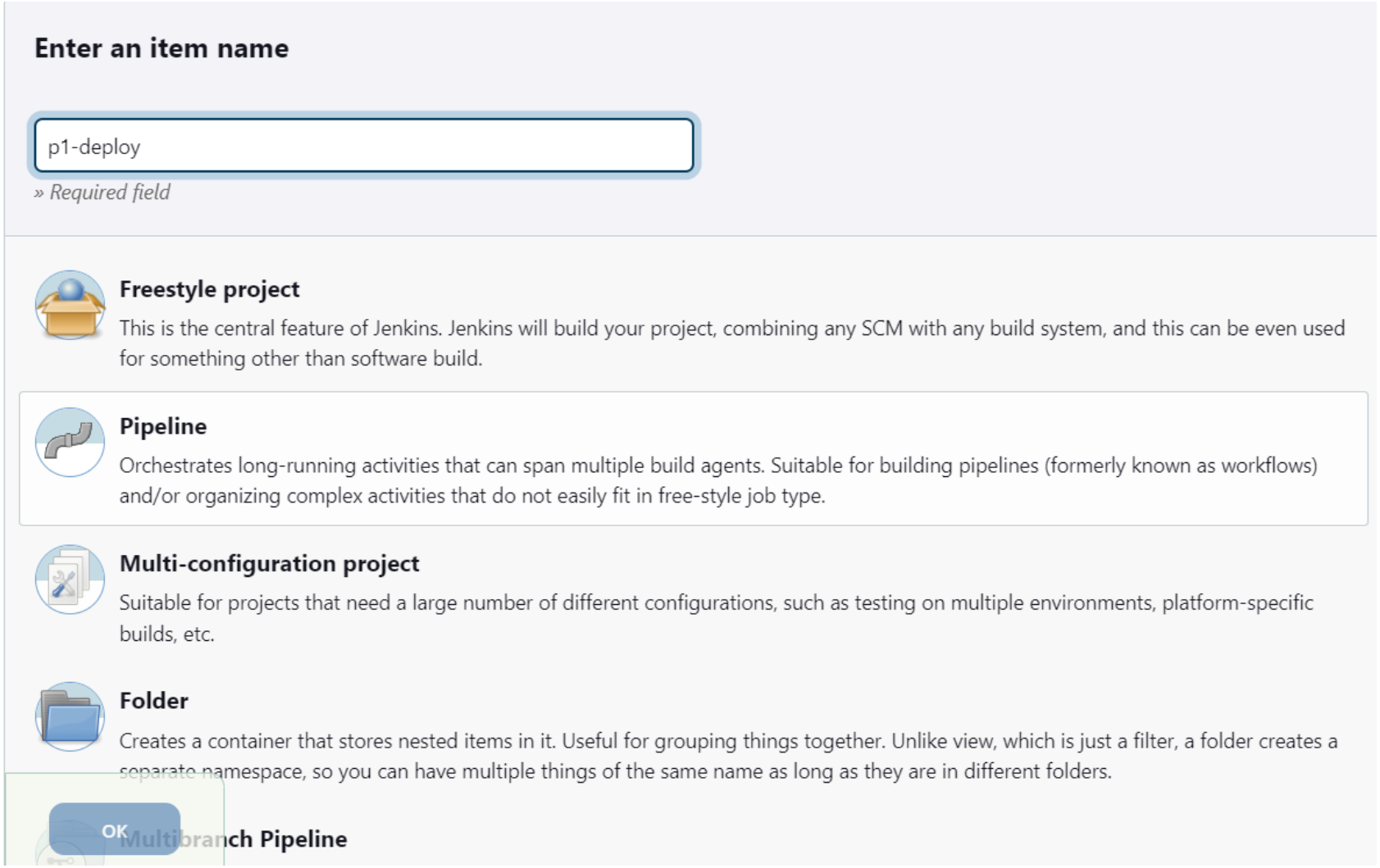


Type following

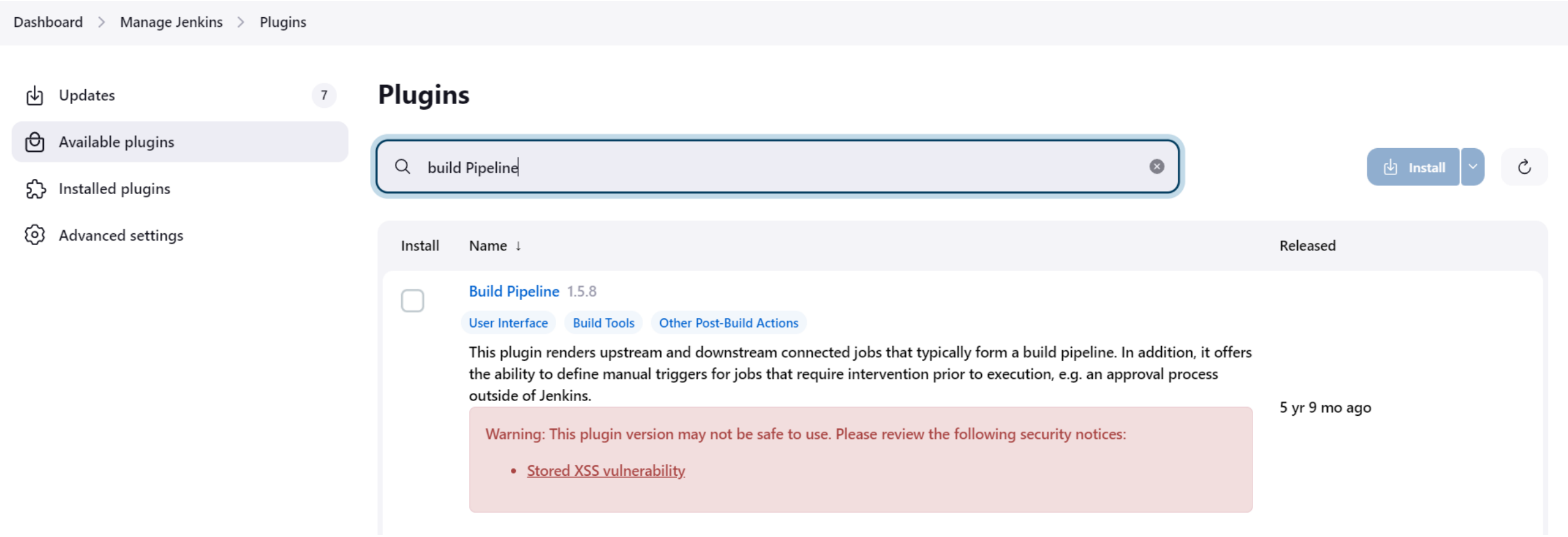
sleep 5
echo "building p1"



Create job 2 (deploy)
****Freestyle project**

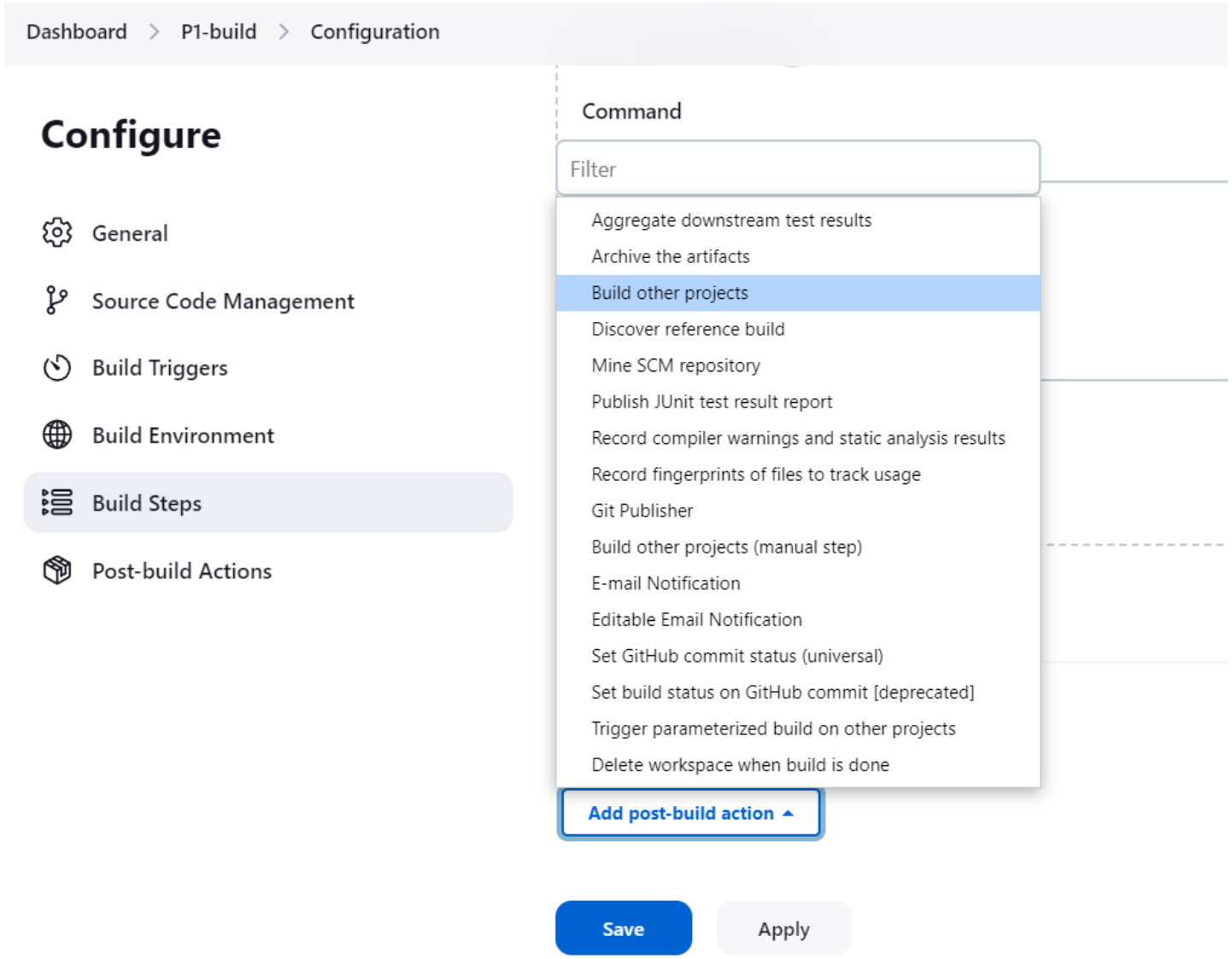


Goto Dashboard->Manage Jenkins->Plugins
Select available plugins
Type build pipeline. The plugin gets visible. Install



Configure jobs to make pipeline

Configure job1 (build)
Post-build Actions-> Build other project

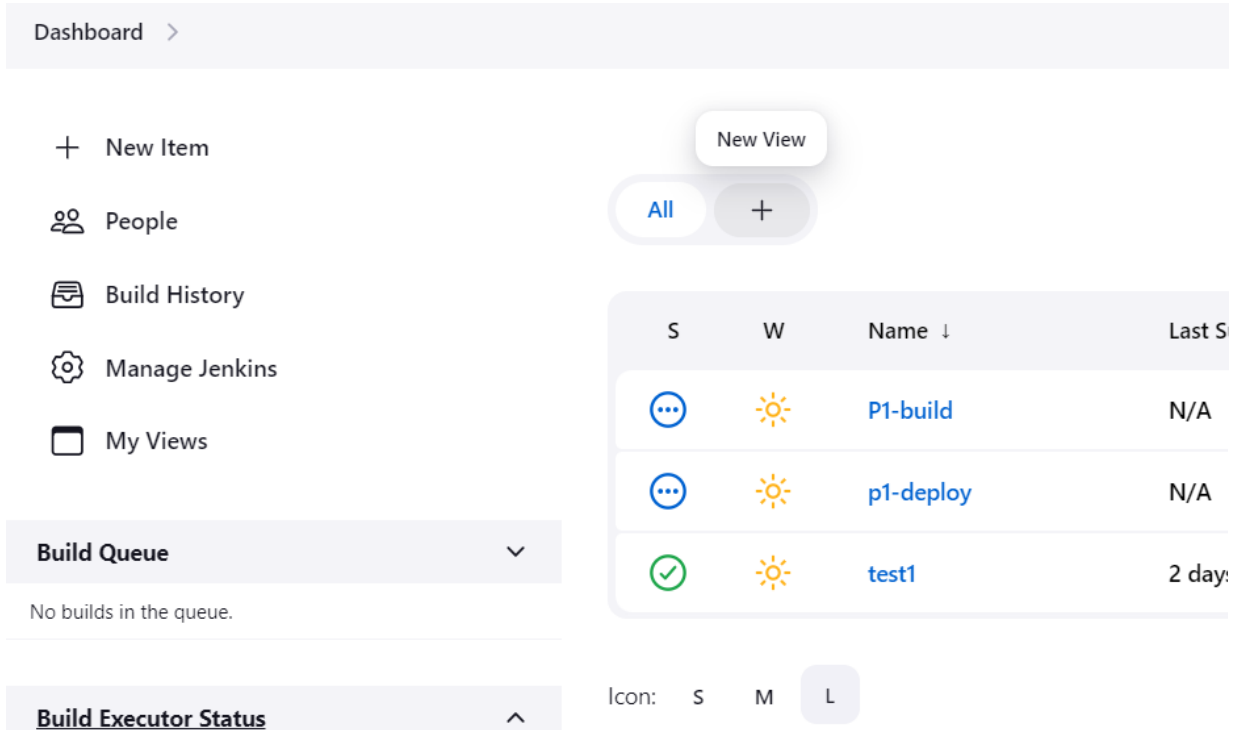


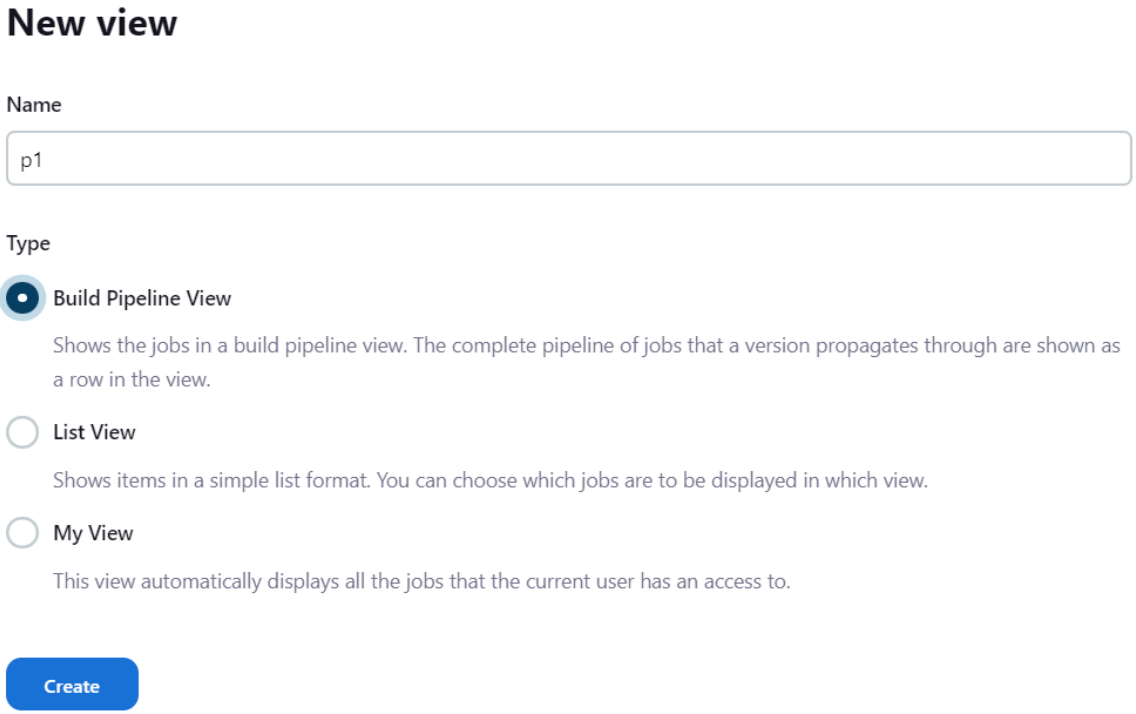
Type the title of job2 (deploy)
Apply and save



View of pipelines

+ sign


Dashboard
New Item
People
Build History
Manage Jenkins
My Views
Build Queue
No builds in the queue.
Build Executor Status
built-in node + 1 agent (0 of 2 executors busy)
New View
All
S
W
Name ↓
Last S
P1-build
N/A
p1-deploy
N/A
test1
2 day
Icon: S M L


New view
Name
p1
Type
Build Pipeline View
Shows the jobs in a build pipeline view. The complete pipeline of jobs that a version propagates through are shown as a row in the view.
List View
Shows items in a simple list format. You can choose which jobs are to be displayed in which view.
My View
This view automatically displays all the jobs that the current user has an access to.
Create

Move to “select initial job”

Pipeline Flow

Layout

Based on upstream/downstream relationship

This layout mode derives the pipeline structure based on the upstream/downstream trigger relationship between jobs. This is the only out-of-the-box supported lay
but is open for extension.

Upstream / downstream config

Select Initial Job ?

P1-build

Trigger Options

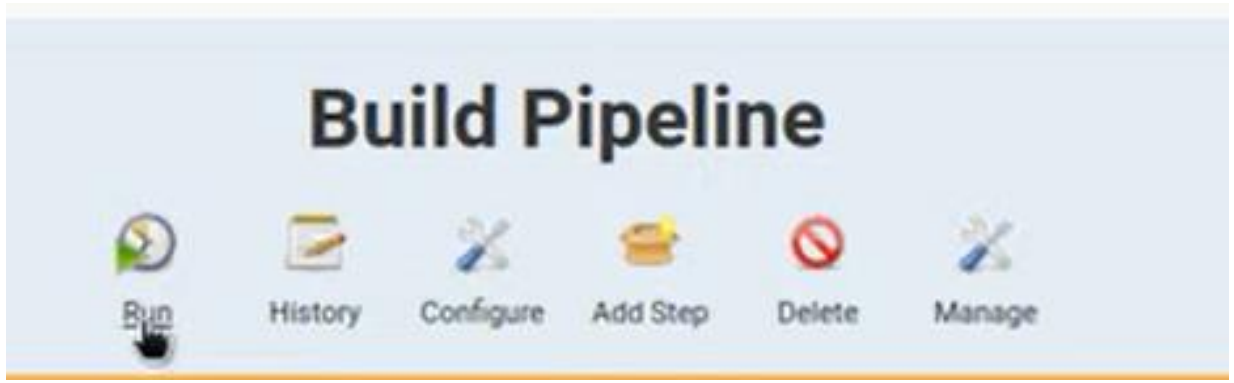
Build Cards

Standard build card

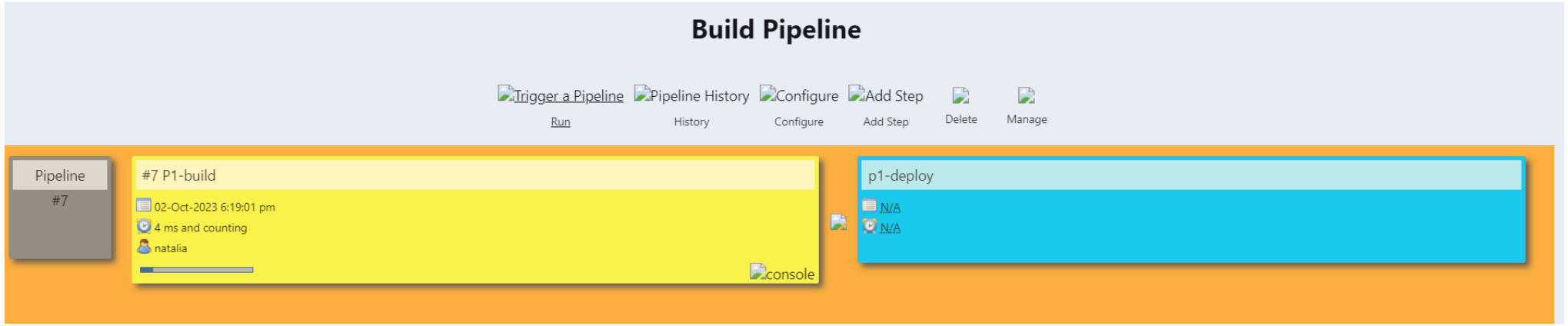
Use the default build cards

Restrict triggers to most recent successful builds ?

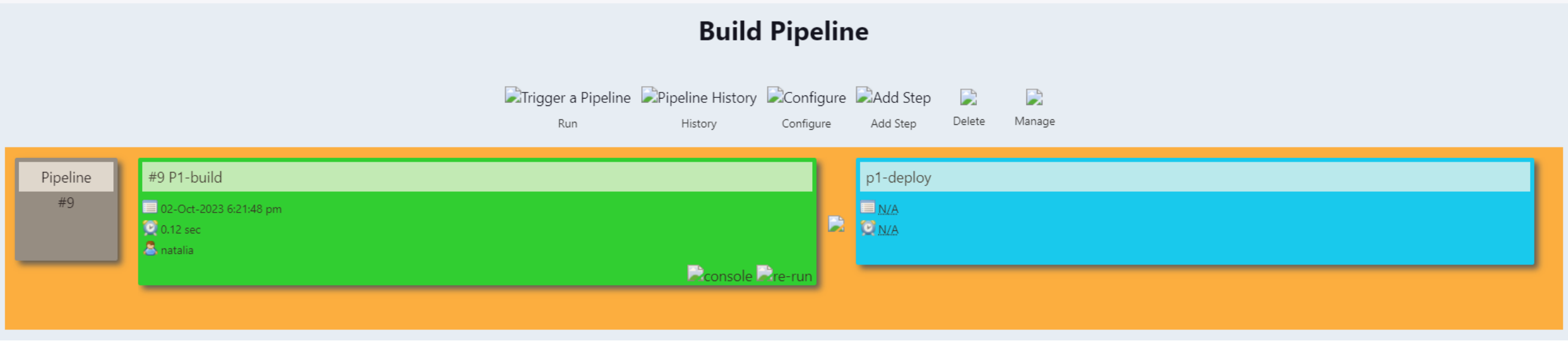
OK Apply


Build Pipeline
Run
History
Configure
Add Step
Delete
Manage

Press Run


Build Pipeline
Trigger a Pipeline
Run
Pipeline History
History
Configure
Configure
Add Step
Add Step
Delete
Manage
Pipeline
#7
#7 P1-build
02-Oct-2023 6:19:01 pm
4 ms and counting
natalia
console
p1-deploy
N/A
N/A

Yellow means running
green means finished running



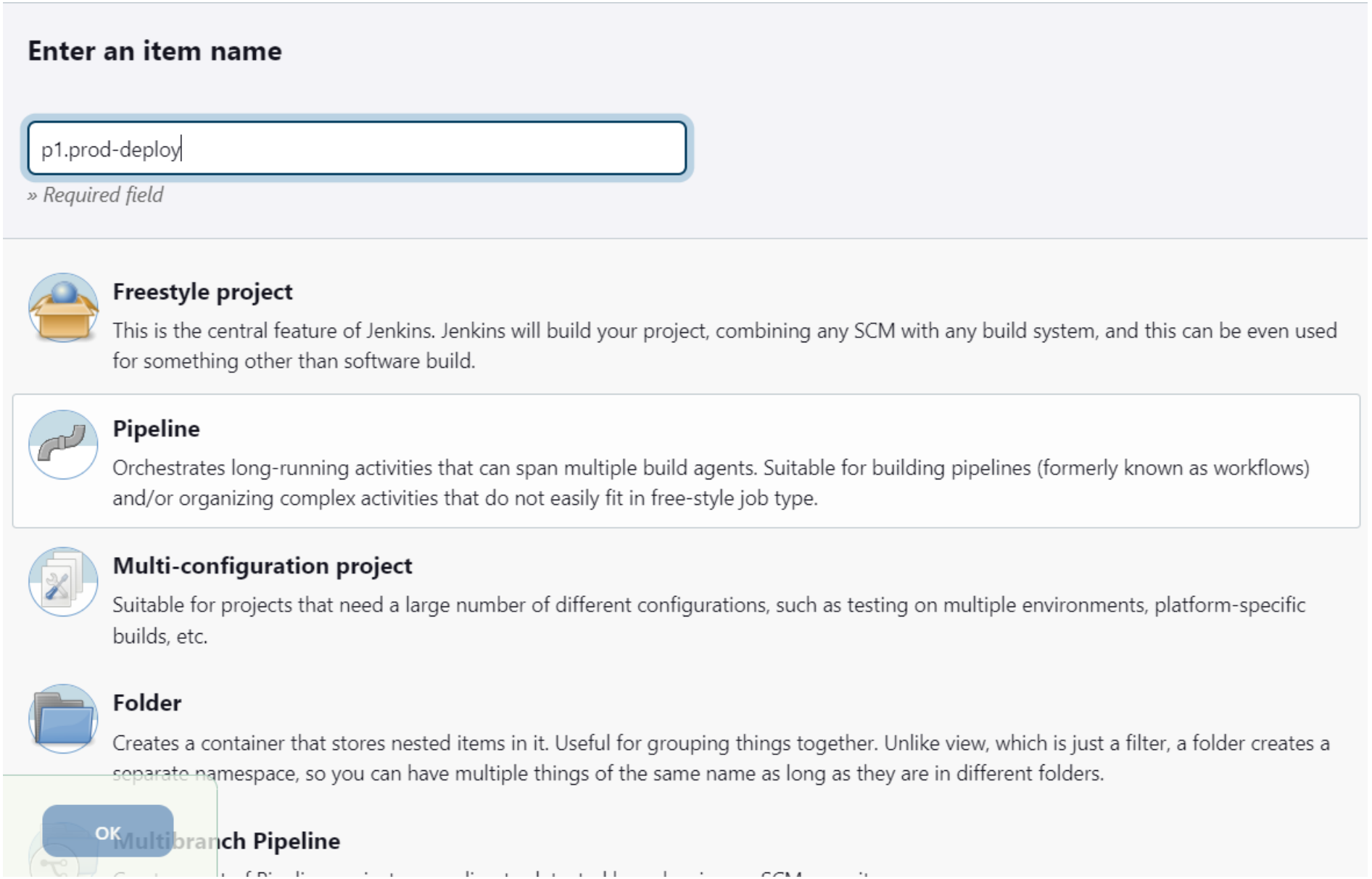
If build fails, set shell path using steps

In Manage Jenkins → Configure System → Shell, set the shell path as

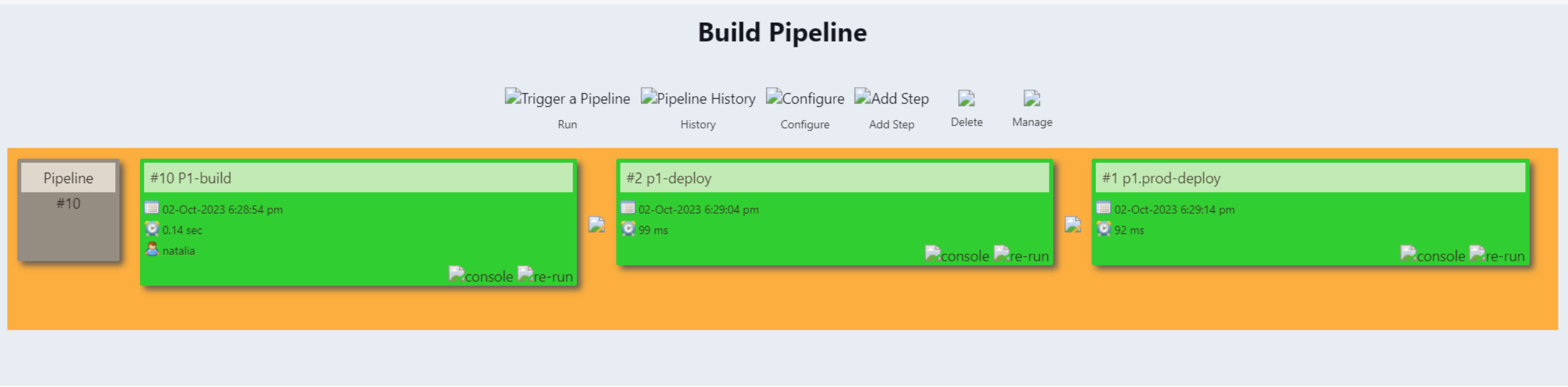
C:\Windows\system32\cmd.exe

Demo Continuous delivery and deployment

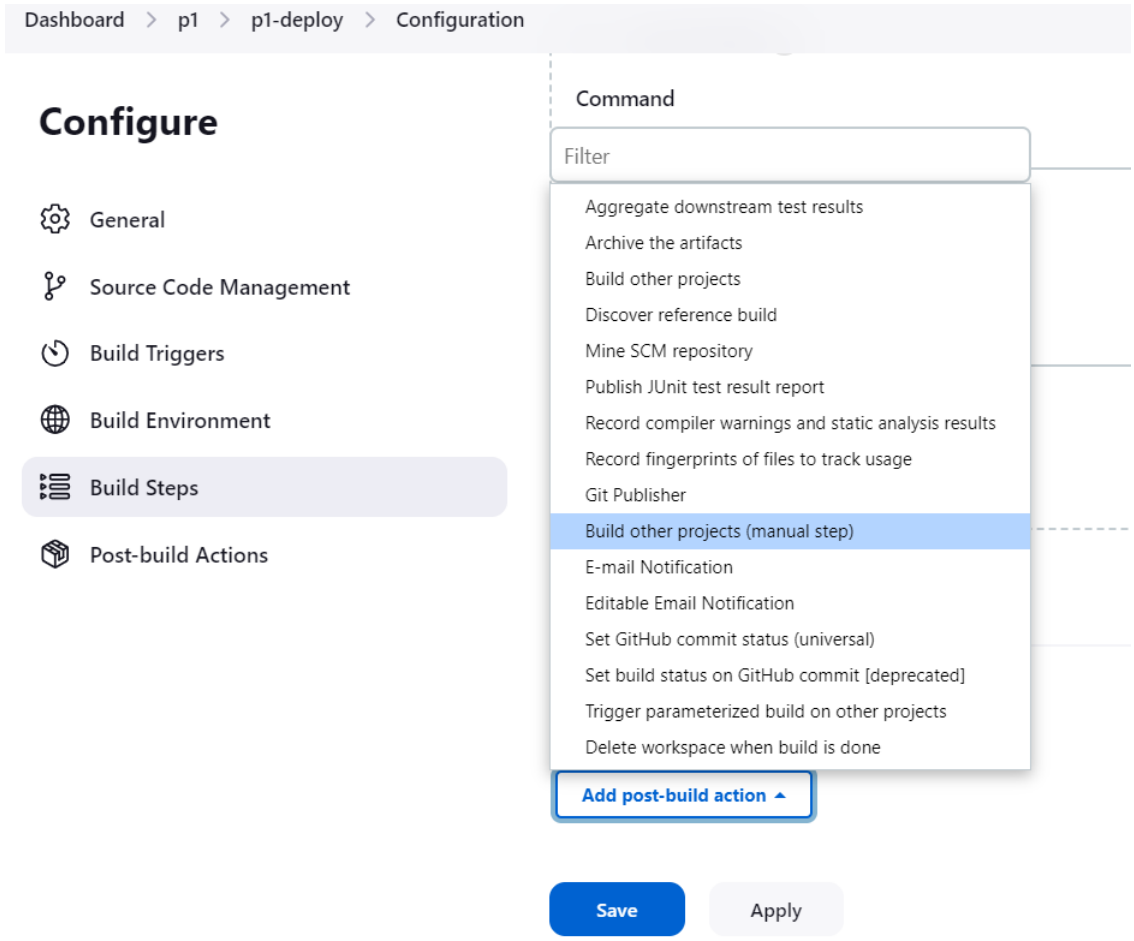
Again create a job and link it with last ceated job (job 2)



Continuous deployment achieved



To achieve continuous delivery…edit job 2 (deploy) and select “build manual” and set downstream project to third job



Pipeline (Summarized)

- Build Pipeline plugin are collection of Jenkins jobs. Jobs are chained together. E.g.

Job 1: build, job 2: test, job 3: deploy

Limitations: complex pipelines hard to manage, maintenance cost increases with the increase in number of processes, good for small count of jobs

Solution: Jenkins pipeline

- Jenkins pipeline: single platform that runs entire pipeline as a code
- suite of plugins which supports implementing and integrating *continuous delivery pipelines* into Jenkins.
- The definition of a Jenkins Pipeline is written into a text file (called a Jenkinsfile)

Declarative versus Scripted Pipeline syntax
A Jenkinsfile can be written using two types of syntax - Declarative and Scripted.

Declarative and Scripted Pipelines are constructed fundamentally differently.
Declarative Pipeline is a more recent feature of Jenkins Pipeline which:

provides richer syntactical features over Scripted Pipeline syntax, and

is designed to make writing and reading Pipeline code easier.

Key features of Jenkins Pipeline

- Code can be checked into a version control system
- Incorporate user input (users can interact, restart from saved checkpoint)
- Runs the jobs in parallel

You can create pipeline files using two main approaches: Declarative and Scripted pipelines.

1. Declarative Pipeline:
Declarative pipelines are recommended for most use cases as they provide a simpler and more concise way to define your CI/CD process.

```
pipeline {
    agent any // Specifies where the pipeline should run (e.g., any available agent)

    stages {
        stage('Checkout') {
            steps {
                // Checkout your source code from a version control system (e.g., Git)
                checkout scm
            }
        }
        stage('Build') {
            steps {
                // Define your build steps (e.g., using Maven or Gradle)
                sh 'mvn clean package'
            }
        }
        stage('Test') {
            steps {
                // Define your testing steps (e.g., running unit tests)
                sh 'mvn test'
            }
        }
        stage('Deploy') {
            steps {
                // Define deployment steps (e.g., deploying to a server)
                sh './deploy-script.sh'
            }
        }
    }
}
```

Here's how you can create a declarative pipeline file:

1. Open your text editor or integrated development environment (IDE).

2. Create a new file with the extension .groovy (e.g., Jenkinsfile.groovy).

3. Copy and paste the above code into the file.

4. Customize the stages and steps to match your specific CI/CD process, including source code management, build tools, testing, and deployment.

5. Save the file.

6. You can then configure your Jenkins job to use this pipeline file by specifying its location in your Jenkins job configuration.

Scripted pipelines provide more fine-grained control and flexibility but can be more complex to write.

```
node {
    try {
        stage('Checkout') {
            checkout scm
        }

        stage('Build') {
            sh 'mvn clean package'
        }

        stage('Test') {
            sh 'mvn test'
        }

        stage('Deploy') {
            sh './deploy-script.sh'
        }
    } catch (Exception e) {
        currentBuild.result = 'FAILURE'
        throw e
    } finally {
        // Post-build cleanup or notifications can be added here
    }
}
```

To create a scripted pipeline file:

1.Follow steps 1 and 2 as mentioned above.

2.Copy and paste the above code into the file.

3.Customize the stages and steps to match your specific CI/CD process, including source code management, build tools, testing, and deployment.

4.Save the file.

5.Configure your Jenkins job to use this pipeline file by specifying its location in your Jenkins job configuration.

Once you've created and configured your pipeline file, Jenkins will automatically execute the defined steps whenever you trigger the job, allowing you to automate your CI/CD process as code. Make sure to test and iterate on your pipeline as needed to ensure it meets your requirements.

- **Pipeline block** is a user defined block which contains all the stages
- **Agent** instructs Jenkins to allocate an executor for the builds
- **Stages** contains all the work, each stage performs a specific task

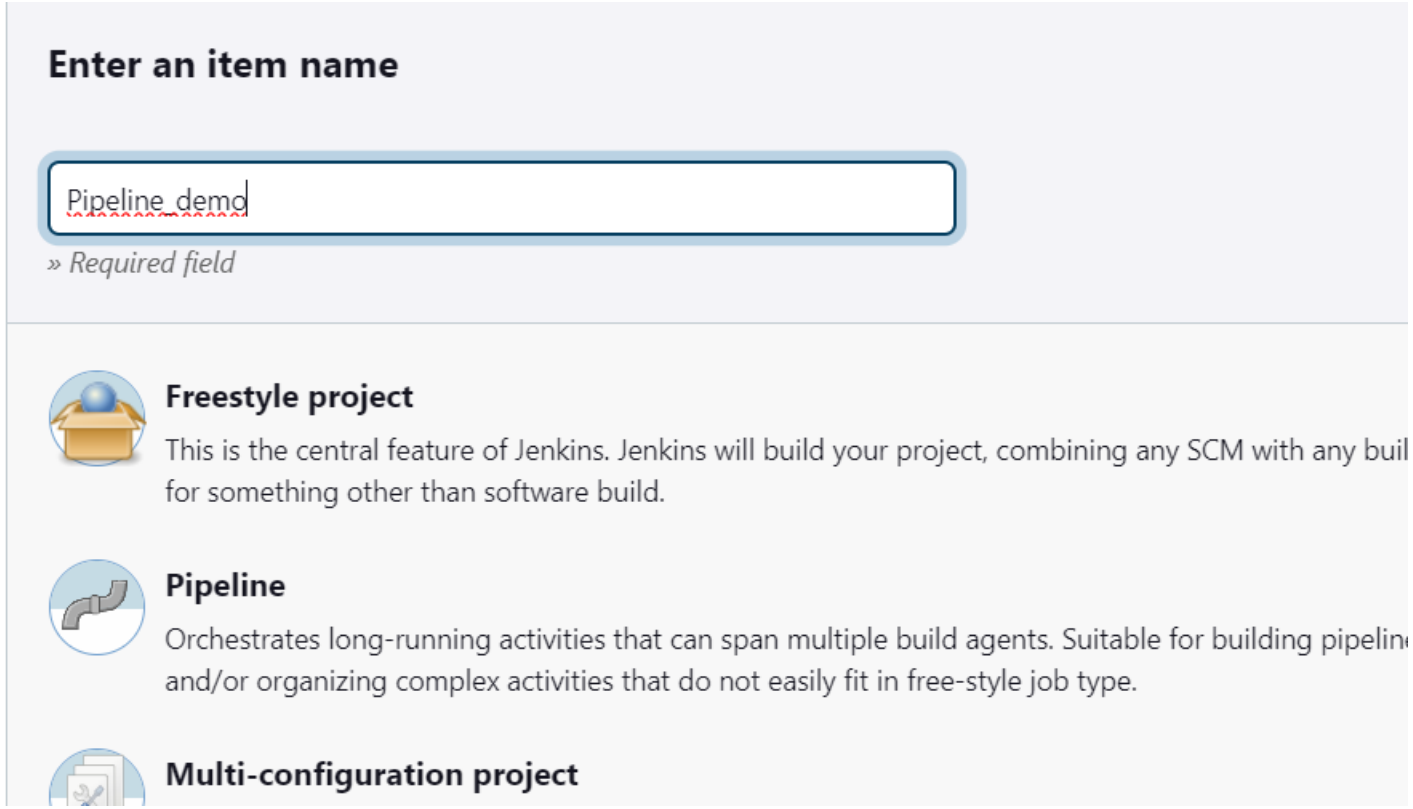
```
Pipeline{
        agent any
                stages
                {
                        stage (‘build’)
                        {
                        }
                        stage (‘test’)
                        {
                        }
                        stage (‘deploy’)
                        {
                        }

                }

}
```

Create pipeline

Dashboard-> New item-> Pipeline



Dashboard > Pipeline_demo > Configuration

Configure

- General
- Advanced Project Options
- Pipeline

General

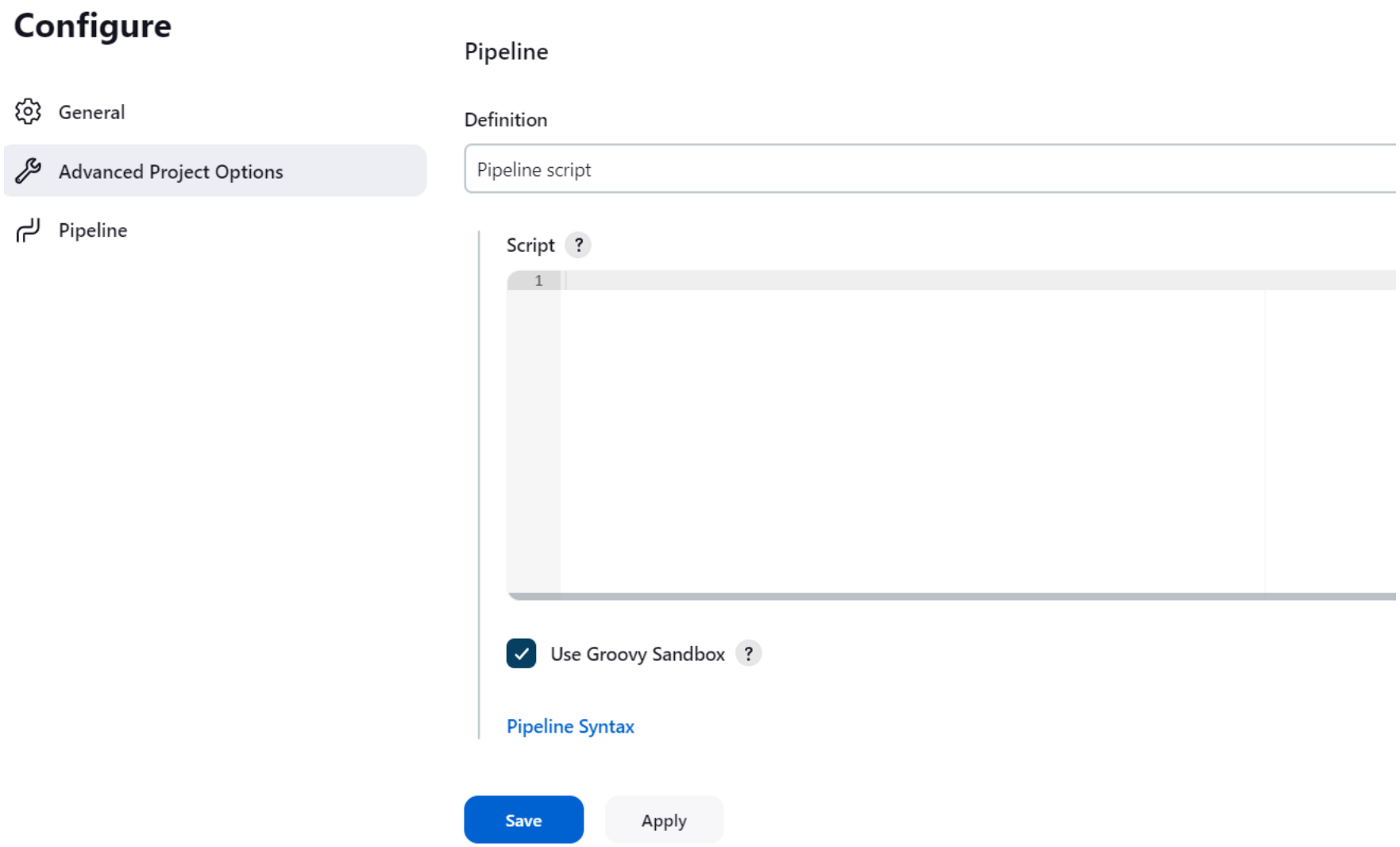
Description

Plain text Preview

- [ ] Discard old builds ?
- [ ] Do not allow concurrent builds
- [ ] Do not allow the pipeline to resume if the controller restarts
- [ ] GitHub project
- [ ] Pipeline speed/durability override ?
- [ ] Preserve stashes from completed builds ?

Save Apply

Move to pipeline section



Configure

- General
- Advanced Project Options
- Pipeline

Pipeline

Definition

Pipeline script

Script ?

1

try sample Pipeline...

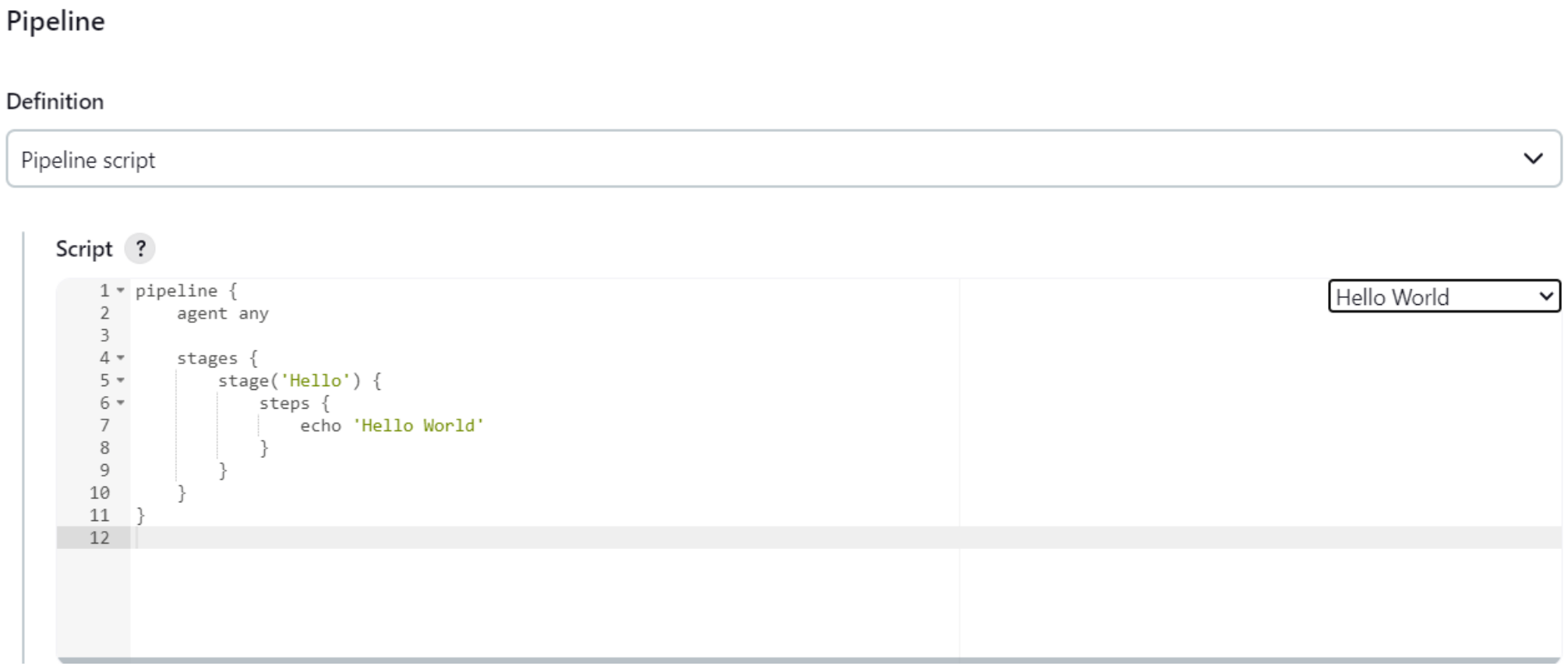
- try sample Pipeline...
- Hello World
- GitHub + Maven
- Scripted Pipeline

Use Groovy Sandbox ?

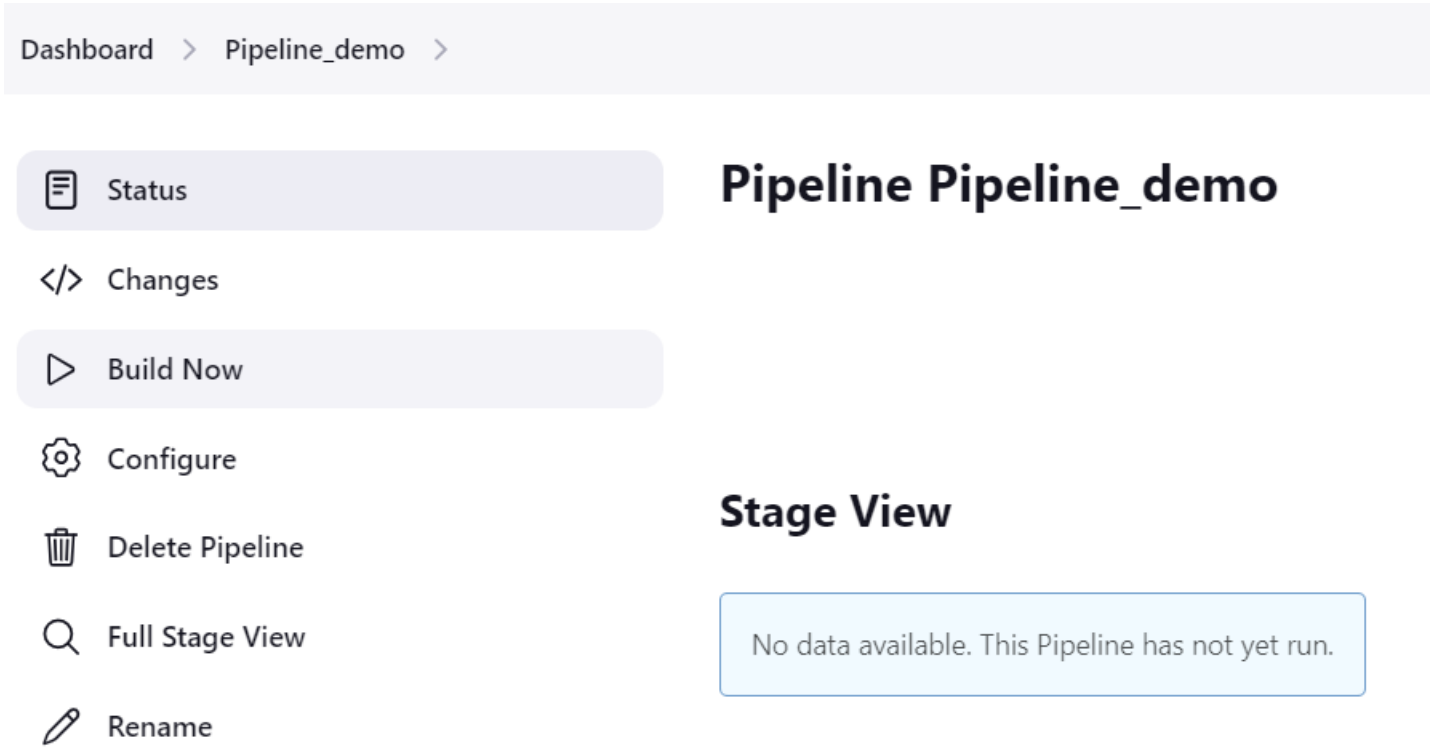
Pipeline Syntax

Save Apply

Agents and stages are known as sections
Echo … are known as declaratives

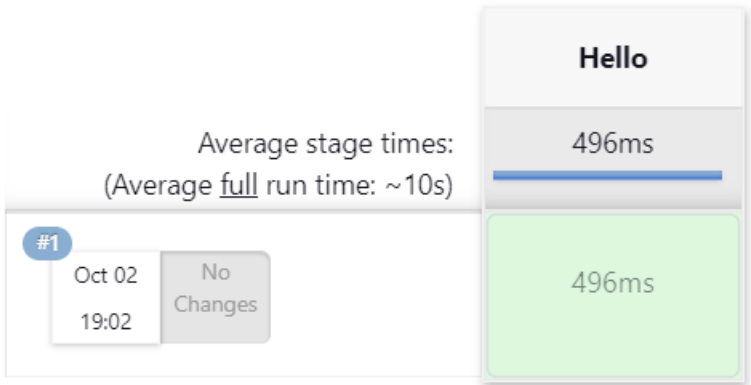


Build now


Dashboard
Pipeline_demo
Status
Changes
Build Now
Configure
Delete Pipeline
Full Stage View
Rename
Pipeline Pipeline_demo
Stage View
No data available. This Pipeline has not yet run.

Pipeline Pipeline_demo

Stage View



Now adding a new job is simply copy pasting a new stage..simple in contrast to basic build pipeline plugin (advantage of Jenkins pipeline (pipeline as a code)

1. Run a command

type n step of a stage
sh “date”

```
pipeline {
    agent any

    stages {
        stage('Run A command') {
            steps {
                sh 'date'

            }
        }
```

2. **Environment variables** in Jenkins are global key-value pairs that can be accessed and injected into a project.

Here are some of the roles of environment variables in Jenkins:

•**Storing sensitive data:** Environment variables can be used to store sensitive data, such as passwords. This data can then be accessed by build scripts or other tools without having to hard code it into the project code.

•**Configuring the build environment:** Environment variables can be used to configure the build environment, such as the path to the source code or the location of the build artifacts.

```
pipeline {
   agent any

   environment {
      MY_SECRET_KEY = "!@#hello#@!"
   }
```

Configure

- General
- Source Code Management
- Build Triggers
- Build Environment
- Build Steps
- Post-build Actions

Plain text Preview

Discard old builds ?

GitHub project

This project is parameterized ?

Add Parameter

Filter

- Boolean Parameter
- Choice Parameter
- Credentials Parameter
- File Parameter
- Multi-line String Parameter
- Password Parameter
- Run Parameter
- String Parameter

ecessary ?

be run ?

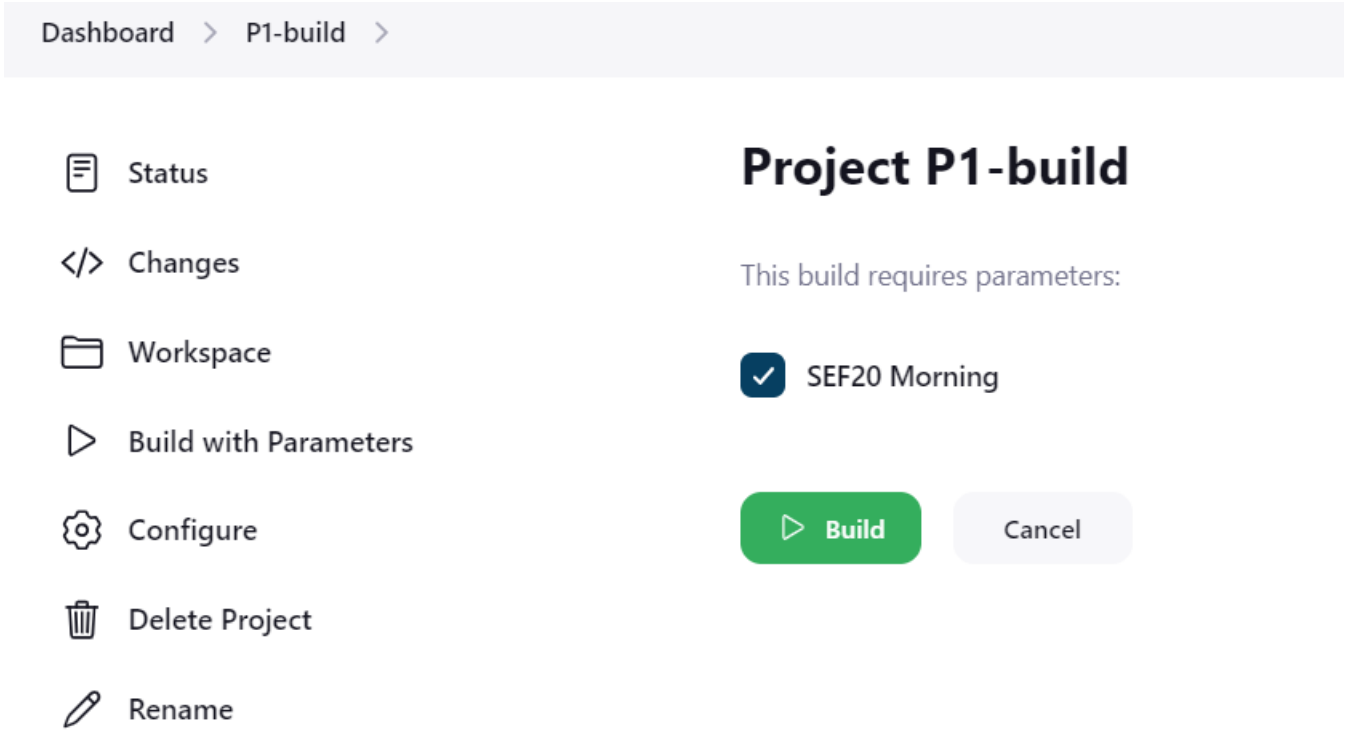
Adv

Source Code Management

None

Save Apply

Boolean parameter added



3. Using Parameters in Jenkinsfile

1.Choice parameters
2.Boolean parameter
3.Multi-line string parameter
4.String Parameter

```
pipeline {
    agent any
    stages {
        stage('Setup parameters') {
            steps {
                script {
                    properties([
                        parameters([
                            choice(
                                choices: ['ONE', 'TWO'],
                                name: 'PARAMETER_01'
                            ),
                            booleanParam(
                                defaultValue: true,
                                description: '',
                                name: 'BOOLEAN'
                            ),
                            text(
                                defaultValue: '''
                                this is a multi-line
                                string parameter example
                                ''',
                                 name: 'MULTI-LINE-STRING'
                            ),
                            string(
                                defaultValue: 'scriptcrunch',
                                name: 'STRING-PARAMETER',
                                trim: true
                            )
                        ])
                    ])
                }
            }
        }
    }
}
```

print "Parameter: ${param}, Value: ${value}"

4. Input

```
stage('Continue ?') {
    input {
        message "Should we continue?"
        ok "Yes we Should"
    }
```

message : String
This parameter gives a prompt which will be shown to a human:

Ready to go?
Proceed or Abort

If you click "Proceed" the build will proceed to the next step, if you click "Abort" the build will be aborted.

Jenkins Input Step Example

```
pipeline {
  agent any
  stages {
    stage ("Prompt for input") {
      steps {
        script {
          env.USERNAME = input message: 'Please enter the username',
                         parameters: [string(defaultValue: '',
                                     description: '',
                                     name: 'Username')]
          env.PASSWORD = input message: 'Please enter the password',
                         parameters: [password(defaultValue: '',
                                     description: '',
                                     name: 'Password')]
        }
        echo "Username: ${env.USERNAME}"
        echo "Password: ${env.PASSWORD}"
      }
    }
  }
}
```

Scripted pipeline

- Scripted Pipeline is the original pipeline syntax for Jenkins, and it is based on Groovy scripting language.
- In Scripted Pipeline, the entire workflow is defined in a single file called a Jenkinsfile. The Jenkinsfile is written in Groovy and is executed by the Jenkins Pipeline plugin.
- Scripted Pipeline provides a lot of flexibility and control over the workflow, but it can be more complex than Declarative Pipeline.

`sh` step to run shell commands

```
node {
stage('Build') {
// Build the application
sh 'mvn clean install'
}
stage('Test') {
// Run the tests
sh 'mvn test'
}
stage('Deploy') {
// Deploy the application
sh 'deploy.sh'
}
}
```

Jenkins scripted pipeline has the following skeleton:

```
node {
        stage (‘Build’ {
                //...
        }
        stage (‘Test’) {
                //...
        }
}
```

The script has the elements “pipeline”, “agent” and “steps” which are specific to Declarative Pipeline syntax; “stage” is common to both Declarative and Scripted; and finally, node” is specific for the Scripted one.

“Pipeline” defines the block that will contain all the script content.
“Agent” defines where the pipeline will be run, similar to the “node” for the scripted one.
“Stages” contains all of the stages.

On the other hand, a declarative pipeline can be written by using more elements, as shown next:

```
pipeline {
        agent any
        stages {
stage(‘Build’) {
        steps {
                //…
        }
        }
        stage (‘Test’) {
        steps {
                //…
        }
        }
}
}
```

Thank you